Apache OpenOffice

Version 4.1

Calc Guide

AOO Documentation Team

Chapter 7

Using Formulas and Functions

Acknowledgments

This document is updated from a previous version by the OOoAuthors team. The AOO Documentation Team appreciates the many contributions of Michael Scarborough and Michele Petrovsky.

Publication date and software version

Published August 25, 2025. Based on Apache OpenOffice 4.1.

Note for Mac Users

Some keystrokes and menu items differ on a Mac from those used in Windows and Linux. The table below gives some common substitutions for the instructions in this chapter. For a more detailed list, see the application Help.

Windows/Linux	***Mac equivalent***	***Effect***
Tools > Options menu selection	**OpenOffice > Preferences**	Access setup options
Right-click	*Control+click*	Open context menu
Ctrl (Control)	⌘ (Command)	Used with other keys
F5	*Shift+⌘+F5*	Open the Navigator
F11	⌘+*T*	Open Styles & Formatting window

Contents

Introduction

In previous chapters, we have entered one of two basic data into each cell: numbers or text. However, we may not always know what the contents should be. Often, one cell's contents depend on the contents of other cells. To handle this situation, we use a third type of data: the *formula*. Formulas are equations that use numbers, text, and variables to get a result. In a spreadsheet, the variables are cell locations that hold the data needed for the equation to be completed.

A *function* is a predefined calculation entered in a cell. Functions help you analyze or manipulate data in a spreadsheet. All you have to do is add the parameters with which the function will work, and the calculation is automatically made for you. Functions help you create the processing needed to get the results that you're looking for.

Setting Up a Spreadsheet

If you are setting up more than a simple one-worksheet system in Calc, it is worth planning ahead. Avoid the following traps:

- Typing fixed values into formulas
- Not including notes and comments describing what the system does, including what input is required and where the formulas come from (if not created from scratch)
- Not incorporating a system of checking to verify that the formulas do what is intended
- Emphasizing a spreadsheet's appearance more than computational simplicity and flexibility

The Trap of Fixed Values

Many users set up long and complex formulas with fixed values typed directly into the formula.

For example, conversion from one currency to another requires knowledge of the current conversion rate. If you input a formula in cell C1 of =0.75*B1 (for example, to calculate the value in Euros of the USD dollar amount in cell B1), you will have to edit the formula when the exchange rate changes from 0.75 to some other value. It is much easier to set up an input cell with the exchange rate and reference that cell in any formula needing the exchange rate. *What-if* type calculations are also simplified: what if the exchange rate varies from 0.75 to 0.70 or 0.80? No formula editing is needed, and it is clear what rate is used in the calculations. Breaking complex formulas down into more manageable parts, described below, also helps to minimize errors and aid troubleshooting.

Lack of Documentation

Lack of adequate documentation happens frequently. Many users prepare a simple worksheet that evolves into something much more complicated over time. Without documentation, the original purpose and methodology can be difficult to decipher. In such cases, it's often easier to start again from the beginning, wasting work done previously. If you insert comments in cells and use labels and headings, you or others can later modify a spreadsheet, saving much time and effort.

Error-checking Formulas

Adding up columns of data or selections of cells from a worksheet often results in errors due to omitting cells, specifying a range incorrectly or double-counting cells. It's useful to institute checks in your spreadsheets. For example, set up a spreadsheet to calculate columns of figures and use SUM to calculate the individual column totals. You can check the result by including (in a non-printing column) a set of row totals and adding these together. The two figures—row total and column total—must agree. If they don't, you have an error somewhere.

	Error Checking Demonstration				
	Sum columns A, B and C				
	A	**B**	**C**		*Row Sums*
	0	0.64	0.02		*0.66*
	0.43	0.23	0.75		*1.41*
	0.91	0.57	0.59		*2.07*
	0.07	0.07	0.45		*0.59*
	0.37	0.33	0.04		*0.74*
	0.34	0.06	0.98		*1.38*
	0.95	0.34	0.65		*1.94*
	0.93	0.08	0.63		*1.64*
	0.61	0.82	0.17		*1.6*
Column Sums	**=SUM(B22:B29)**		**4.26**		
		TOTAL:	**11.37**		*12.03*
					ERROR!!!

Figure 1: Error checking of formulas

You can even set up a formula to calculate the difference between the two totals and report an error if a non-zero value results, i.e. a result in which there is a difference. (see Figure 1).

Appearance vs. Computation

The most visually appealing and human-readable arrangement of data is often very different than the arrangement that allows the most efficient and flexible computations. The visual clues that help people, such as physical separation between different categories of data or changes to fonts or colors, are irrelevant or even troublesome to the functions that do the calculations. For example, imagine you have one year of data, and want to provide a monthly summary. Many people would place the data for each month on a separate sheet, especially if the monthly data and summary would then fit on the screen. The sheet tabs would then provide easy navigation, and the data and summary could have an appealing, uncluttered layout. However, if a new question is asked of the data, the monthly separation may prove inconvenient. If asked to compare weekday versus weekend data, you would have to write complicated formulas or rearrange the data. A simpler layout with all of the data on one sheet with the date in one column and the measured values in adjacent columns would be less visually appealing but would allow you to use the full power of the functions and tools in Calc. Visually appealing summaries can be made while preserving computational ease. Every situation has its own requirements, but it is usually best to keep data together and use the power of Calc to extract subsets or summaries.

Creating Formulas

You can enter formulas in two ways: directly into the cell itself or at the input line. Either way, you need to start a formula with one of the following symbols: =, +, or –. Starting with anything else causes the formula to be treated as if it were text.

Operators in Formulas

Each cell on the worksheet can be used either to hold data, or as a place for data calculations. Entering data is accomplished simply by typing in the cell and moving to the next cell or pressing *Enter*. With formulas, the equals sign indicates that the cell will be used for a calculation. A mathematical calculation like 15 + 46 can be accomplished, as shown in Figure 2.

While the calculation on the left was accomplished in only one cell, the real power is shown on the right, where the data is placed in cells, and the calculation is performed using references back to the cells. In this case, cells B3 and B4 were the data holders, with B5 the cell where the calculation was performed. Notice that the formula was shown as =B3+B4. The plus sign indicates that the contents of cells B3 and B4 are to be added together and then have the result in the cell holding the formula. All formulas build upon this concept. Examples of formulas are shown in Table 1.

These cell references allow formulas to use data from anywhere in the worksheet being worked on or from any other worksheet in the open spreadsheet. If the data needed were in different worksheets, they would be referenced by referring to the name of the worksheet, for example, `=SUM(Sheet2.B12+Sheet3.A11)`.

Note	To enter the = symbol for a purpose other than creating a formula as described in this chapter, type an apostrophe or single quotation mark before the =. For example, in the entry '= *means different things to different people*, Calc treats everything after the single quotation mark—including the = sign—as text.

Simple Calculation in 1 Cell	Calculation by Reference

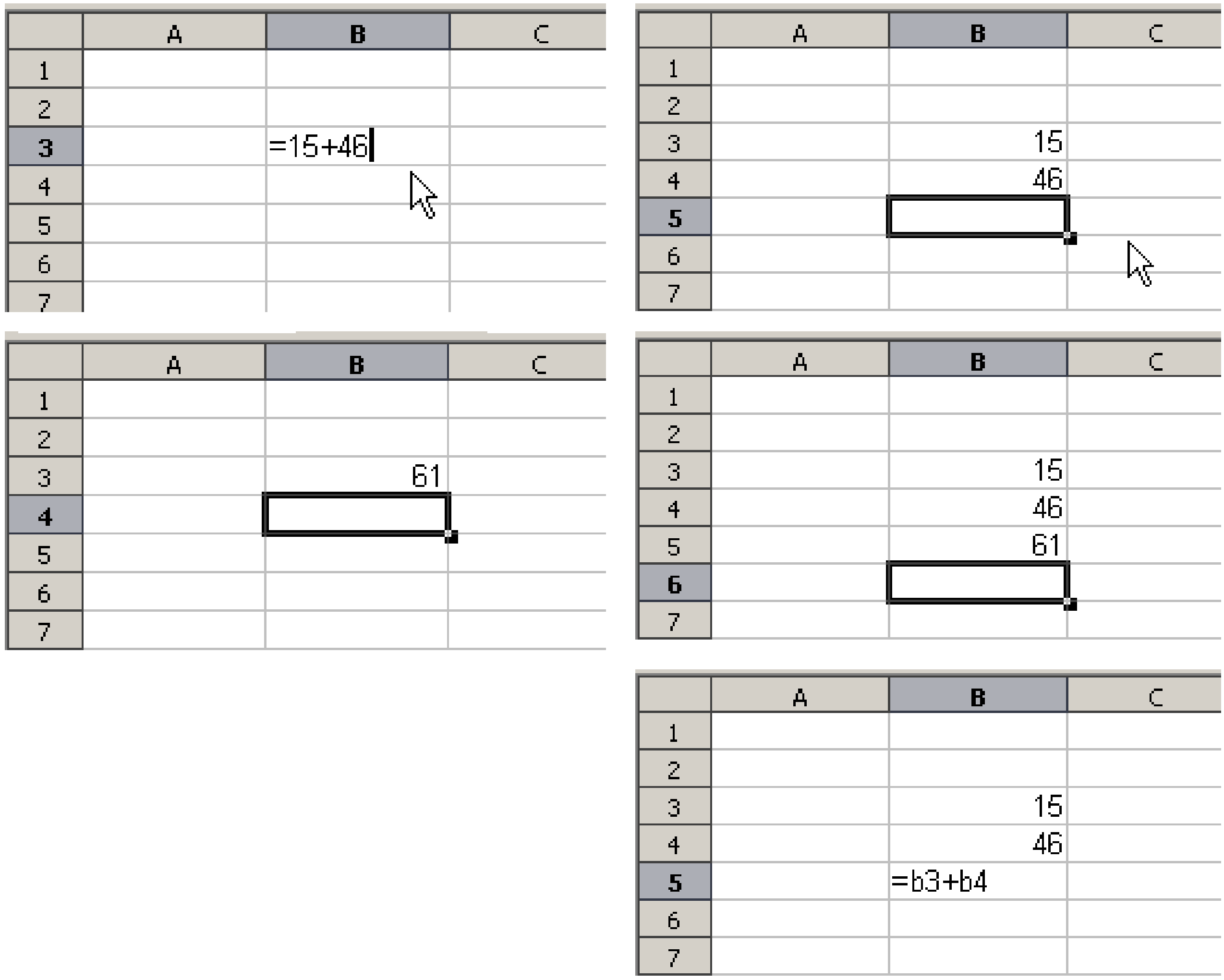


Figure 2: A simple calculation

Table 1: Examples of formulas

Formula	Description
=A1+10	Displays the contents of cell A1 plus 10.
=A1*16%	Displays 16% of the contents of A1.
=A1*A2	Displays the result of the multiplication of A1 and A2.
=ROUND(A1;1)	Displays the contents of cell A1 rounded to one decimal place.
=EFFECTIVE(5%;12)	Calculates the effective interest for 5% annual nominal interest with 12 payments a year.
=B8-SUM(B10:B14)	Calculates B8 minus the sum of the cells B10 to B14.
=SUM(B8;SUM(B10:B14))	Calculates the sum of cells B10 to B14 and adds the value to B8.

Formula	Description
=SUM(B1:B1048576)	Sums all numbers in column B.
=AVERAGE(BloodSugar)	Displays the average of a named range defined under the name BloodSugar.
=IF(C31>140; "HIGH"; "OK")	Displays the results of a conditional analysis of data from two sources. If the contents of C31 are greater than 140, then HIGH is displayed; otherwise, OK is displayed.

Functions can be identified in Table 1 with a word, for example, ROUND, followed by parentheses enclosing parameters fed to the function.

It is also possible to establish ranges for inclusion by naming them using **Insert > Names**, for example, BloodSugar representing a range such as B3:B10. Logical functions can also be performed as represented by the IF statement. Such a use results in a conditional response based on the data in the identified cell. For example:

```
=IF(A2>=0;"Positive";"Negative")
```

produces the display *Positive* if the value in cell A2 is 3 but *Negative* if the value there is –9, or any other value less than 0.

Operator Types

You can use the following operators in OpenOffice Calc: arithmetic, comparative, descriptive, text, and reference.

Arithmetic Operators

Addition, subtraction, multiplication, and division operators return numerical results. The negation and percent operators identify a characteristic of the number found in the cell, for example, -37. Exponentiation illustrates how to enter a number multiplied by itself a certain number of times, for example, 2^3 = 2*2*2.

Table 2: Arithmetical operators

Operator	Name	Example
+ (Plus)	Addition	=1+1
– (Minus)	Subtraction	=2–1
– (Minus)	Negation	–5
* (asterisk)	Multiplication	=2*2
/ (Slash)	Division	=10/5
% (Percent)	Percent	15%
^ (Caret)	Exponentiation	2^3

Comparative Operators

Comparative operators result in either a true or false answer. An example using the IF function could be `=IF(B6>G12; 127; 0)`, which, loosely translated, means if the contents of cell B6 are greater than those of cell G12, produce the number 127. Otherwise, display the number 0.

A direct answer, TRUE or FALSE, can be obtained by entering a formula such as `=B6>B12`. If the value of B6 is greater than that of B12, TRUE is displayed. Otherwise, FALSE is output.

Table 3: Comparative operators

Operator	***Name***	***Example***
= (equal sign)	Equal	A1=B1
> (Greater than)	Greater than	A1>B1
< (Less than)	Less than	A1<B1
>= (Greater than or equal to)	Greater than or equal to	A1>=B1
<= (Less than or equal to)	Less than or equal to	A1<=B1
<> (Inequality)	Inequality	A1<>B1

Note One of the most common errors in programming, and in calculations in general, is mixing up >=, >, <=, <

If cell A1 contains the numerical value 4 and cell B1 has the numerical value 5, the above examples yield results of FALSE, FALSE, TRUE, FALSE, TRUE, and TRUE.

Text Operators

It is common for users to place text in spreadsheets. Text can be joined together in pieces coming from different places on the spreadsheet to provide for variability in what and how this type of data is displayed. Figure 3 shows an example.

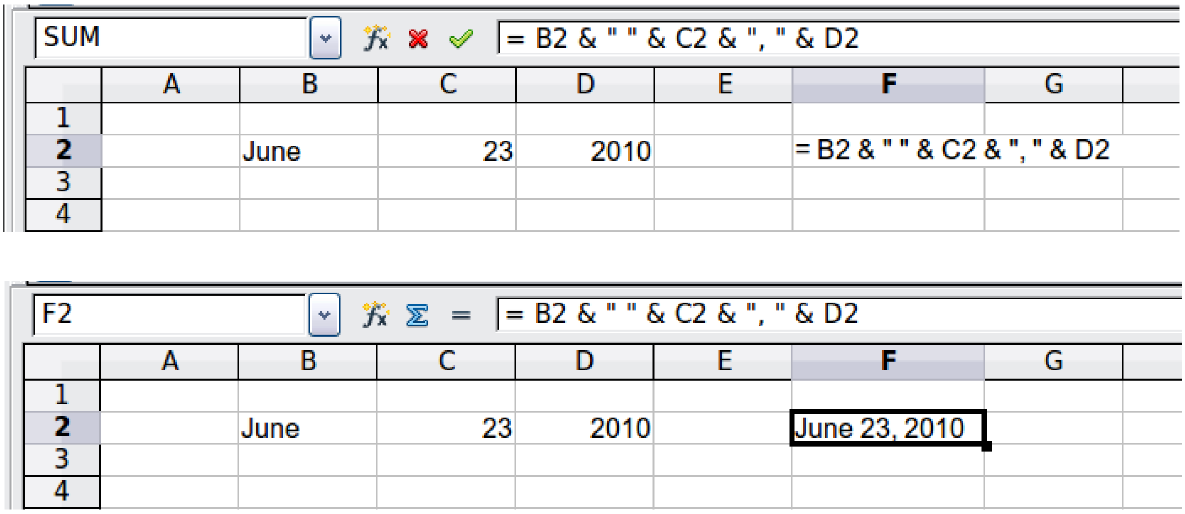


Figure 3: Text concatenation

In this example, specific pieces of the text were found in three different cells. To join these segments, the formula also adds required spaces and punctuation enclosed within quotation marks, resulting in a formula of `=B2 & " " & C2 & ", " & D2`. The result is the concatenation into a date formatted in a particular sequence. The & operator concatenates and glues together pieces of text.

Calc has a CONCATENATE function, which performs the same operation.

Taking this example further, if the result cell is defined as a name, then text concatenation is performed using this defined name. This process is demonstrated in Figures 4, 5, and 6, where the cell with the date is named "WizardDay" and subsequently used in a formula in another cell.

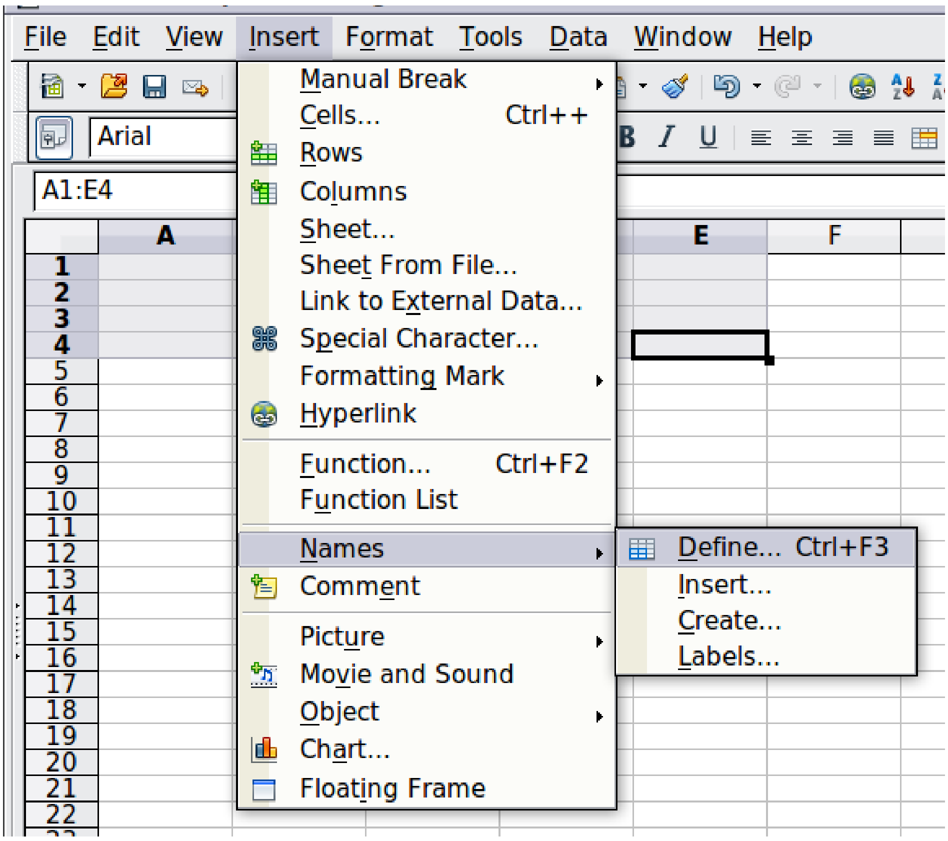


Figure 4: Defining a name for a range of cells

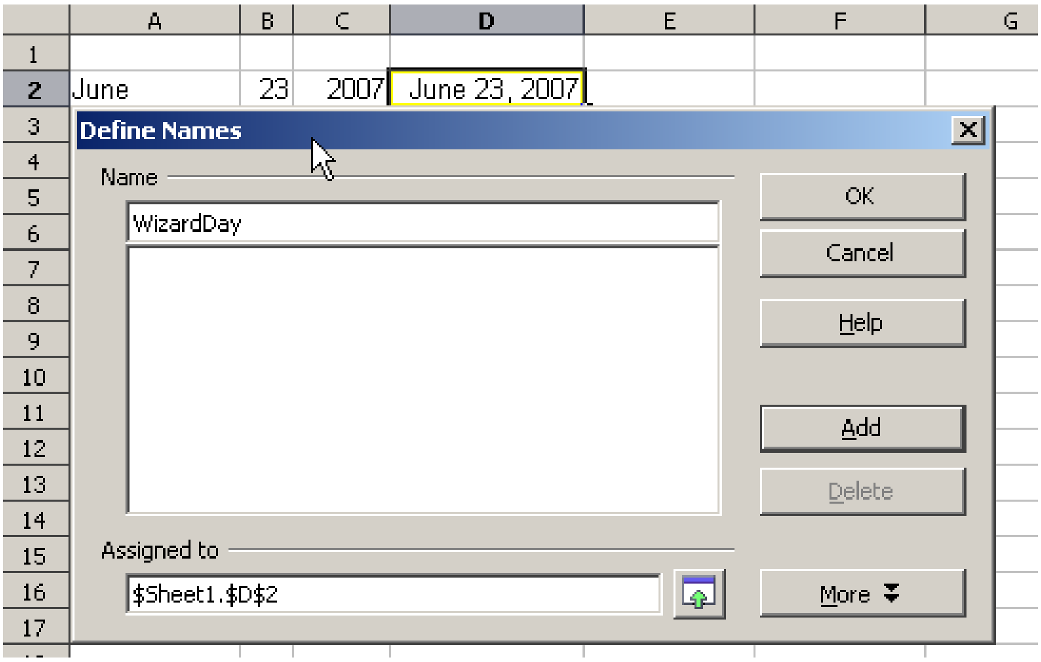


Figure 5: Naming a cell or range of cells for inclusion in a formula

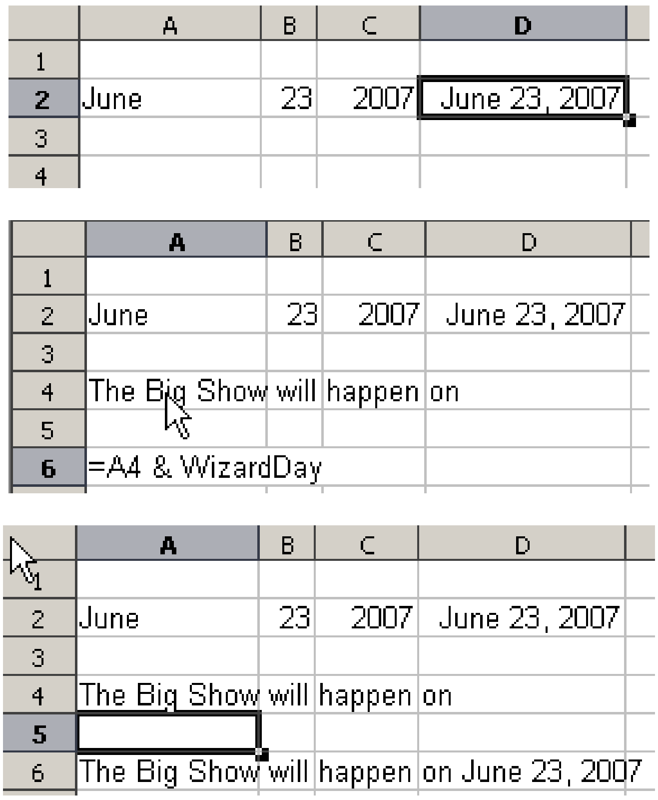


Figure 6: Defining Names on a worksheet

Reference Operators

In its simplest form, a reference points to a single cell, but references can also apply to a rectangle or cuboid as a range or a reference in a list of references. To build such lists, you need reference operators.

An individual cell is identified by the column identifier (letter) located along the top of the columns, and a row identifier (number) found along the left-hand side of the spreadsheet. On spreadsheets read from left to right, the upper left cell is A1.

Range Operator

The range operator is written as a colon. An expression using the range operator has the following syntax:

```
reference left : reference right
```

The range operator builds a reference to the smallest range, including both the cells referenced with the left reference and the cells referenced with the right reference.

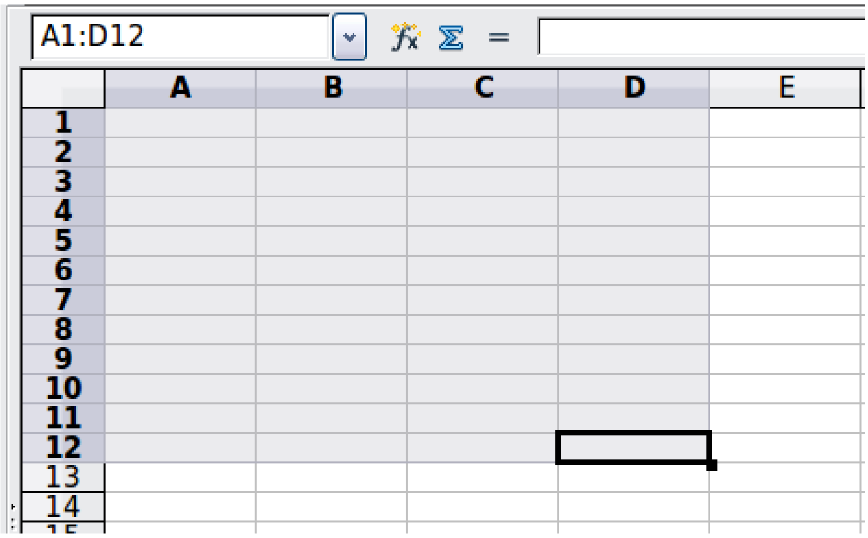


Figure 7: Reference Operator for a range

In the upper left corner of Figure 7, the reference A1:D12 is shown, corresponding to the cells included in the drag operation with the mouse to highlight the range.

Examples

A2:B4	Reference to a rectangle range with 6 cells, 2 column width × 3 row height. When you click on the reference in the formula in the input line, a border indicates the rectangle.
(A2:B4):C9	Reference to a rectangle range with cell A2 top left and cell C9 bottom right. So the range contains 24 cells, 3 column width × 8 row height. This method of addressing extends the initial range from A2:B4 to A2:C9.
Sheet1.A3:Sheet3.D4	Reference to a cuboid range with 24 cells, 4 column width × 2 row height × 3 sheets depth.

When you enter B4:A2 or A4:B2 directly, Calc will turn it into A2:B4, with the left top cell of the range left of the colon and the bottom right cell right of the colon. But if you name the cell B4, for example, as `_start` and A2 as `_end`, you can use `_start:_end` without any error.

Calc, unlike some other spreadsheet programs, cannot reference a whole column of unspecified length using A:A or a whole row using 1:1.

Reference Concatenation Operator

The concatenation operator is written as a tilde. An expression using the concatenation operator has the following syntax:

```
reference left ~ reference right
```

The result of such an expression is an ordered list of references. Some functions, such as SUM, MAX, or INDEX can take a reference list as an argument.

Reference concatenation is sometimes called 'union'. But it's not the union of 'reference left' and 'reference right.' COUNT(A1:C3~B2:D2) displays 12 (=9+3), but it has only 10 cells when considered the union of the two sets of cells.

Intersection Operator

The intersection operator is written as an exclamation mark. An expression using the intersection operator has the following syntax:

```
reference left ! reference right
```

If the references refer to single ranges, the result is a reference to a single range containing all cells in both the left reference and the right reference.

If the references are reference lists, then each list item from the left is intersected with each one from the right, and these results are concatenated into a reference list. The order is to first intersect the first item from the left with all items from the right, then intersect the second item from the left with all items from the right, and so on.

Examples

`A2:B4 ! B3:D6`

This results in a reference to the range B3:B4 because these cells are inside A2:B4 and B3:D4.

`(A2:B4~B1:C2) ! (B2:C6~C1:D3)`

First the intersections A2:B4!B2:C6, A2:B4!C1:D3, B1:C2!B2:C6 and B1:C2!C1:D3 are calculated. This results in B2:B4, empty, B2:C2, and C1:C2. Then, these results are concatenated, dropping empty parts. The final result is the reference list B2:B4 ~ B2:C2 ~ C1:C2.

You can use the intersection operator to refer to a cell in a cross tabulation in an understandable way. If you have columns labeled 'Temperature' and 'Precipitation' and rows labeled 'January,' 'February,' 'March,' and so on, then the following expression

```
'February' ! 'Temperature'
```

will refer to the cell containing the temperature in February.

The intersection operator (!) should have a higher precedence than the concatenation operator (~), but do not rely on precedence. By precedence, we mean the order in which operations are performed within an expression. Some operations, like multiplication and division, are done before addition and subtraction. To enforce another order, we must use parentheses.

Tip	Always put the part that is to be calculated first in parentheses.

Relative and Absolute References

References are the way that we refer to the location of a particular cell in Calc and can be either relative (to the current cell) or absolute (a fixed location).

Relative Referencing

An example of a relative reference will illustrate the difference between a relative reference and absolute reference using the spreadsheet from Figure 8.

1) Type the numbers 4 and 11 into cells C3 and C4, respectively, of that spreadsheet.
2) Copy the formula in cell B5, which is `= B3 + B4`, to cell C5. You can do this by using a simple copy and paste or click and drag B5 to C5, as shown below.
3) Click in cell C5. The formula bar shows =C3+C4 rather than =B3+B4, and the value in C5 is 15, the sum of 4 and 11, which are the values in C3 and C4.

In cell B5, the references to cells B3 and B4 are relative references. This means that Calc interprets the formula in B5 as "sum the cells two and one rows above the cell containing this formula," applies it to the cells in the B column, and puts the result in the cell holding the formula. When you copied the formula to another cell, the same procedure was used to calculate the value to put in that cell. This time, the formula in cell C5 referred to cells C3 and C4.

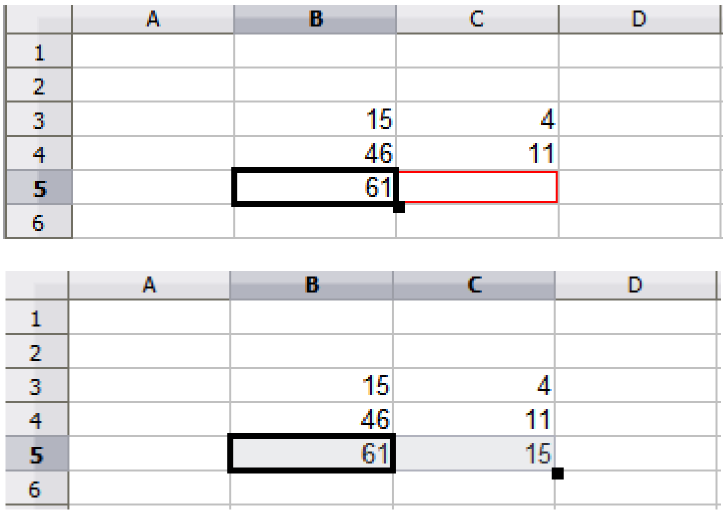


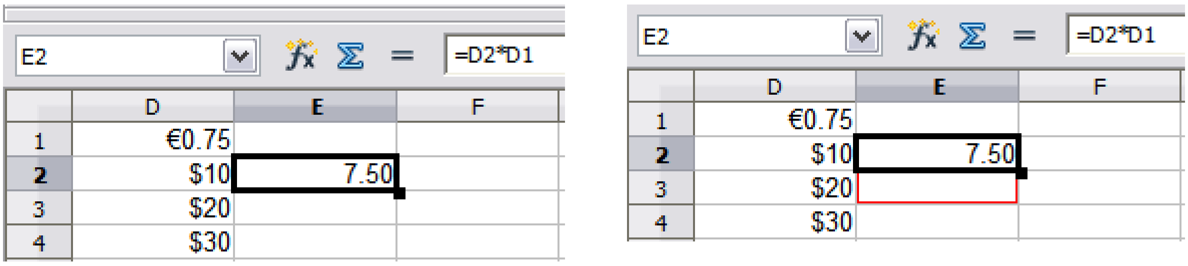
Figure 8: Relative references

Absolute Referencing

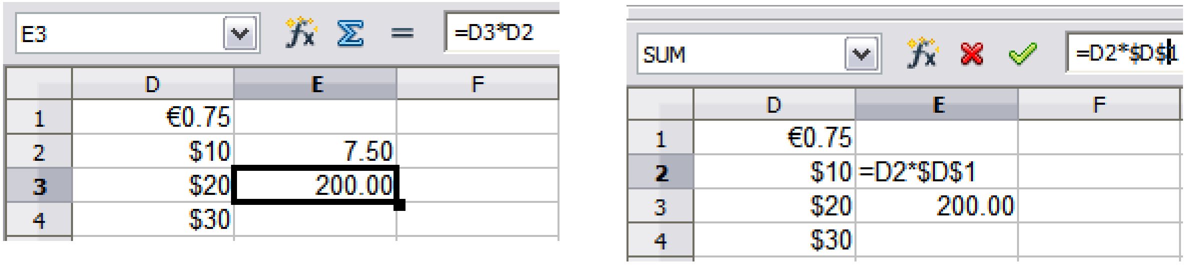
You may want to multiply a column of numbers by a fixed amount. A column of figures might show amounts in US Dollars. To convert these amounts to Euros, it is necessary to multiply each dollar amount by the exchange rate. $US10.00 would be multiplied by 0.75 to convert to Euros, in this case Eur7.50. The following example shows how to input an exchange rate and use that rate to convert amounts in a column from USD to Euros.

1) Input the exchange rate Eur:USD (0.75) in cell D1. Enter amounts (in USD) into cells D2, D3, and D4, for example, 10, 20, and 30.
2) In cell E2, enter the formula =D2*D1. The result is 7.5, correctly shown.
3) Copy the formula in cell E2 to cell E3. The result is 200, clearly wrong! Calc has copied the formula using relative addressing - the formula in E3 is =D3*D2 and not what we want, which is =D3*D1.

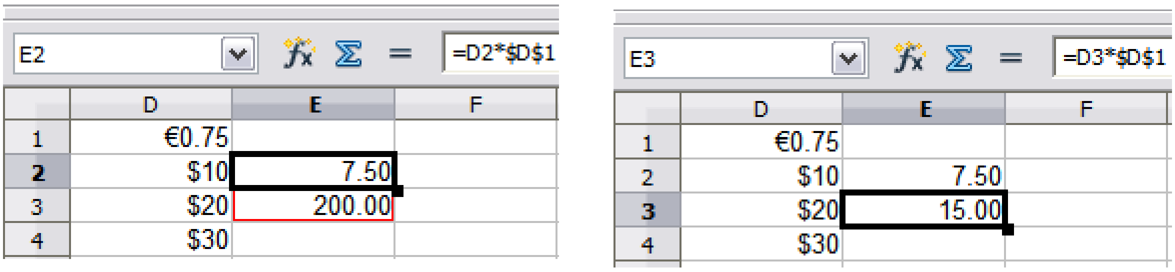
4) In cell E2, edit the formula to be =D2*D1. Copy it to cells E3 and E4. The results are now 15 and 22.5, which are correct.



Steps 2: Setting the exchange rate of Eur at 7.5, then copying it to E3



Copying formula from E2 to E3 & changing the formula to read absolute reference



Applying the correct formula from E2 to E3

Figure 9: Absolute References

The $ signs before the D and the 1 convert the reference to cell D1 from relative to absolute or fixed. If the formula is copied to another cell, the second part will always show D1. The interpretation of the formula `=D2*$D$1` in E2 is "take the value in the cell one column to the left in the same row and multiply it by the value in cell D1".

Cell references can be shown in four ways.

Reference	***Explanation***
D1	Relative, from cell E3 it is the cell one column to the left and two rows above
D1	Absolute, from cell E3, or any other cell, it is the cell D1
$D1	Partially absolute, from cell E3 it is the cell in column D and two rows above
D$1	Partially absolute, from cell E3 it is the cell one column to the left and in row 1

Tip	To change references in formulas, highlight the cell and press *Shift+F4* to cycle through the four different types of references. But be aware that this is of limited value in more complicated formulas since it is usually quicker to edit the formula by hand.

Knowledge of the use of relative and absolute references is essential if you want to copy and paste formulas, and link spreadsheets.

Order of Calculation

Order of calculation refers to the sequence in which numerical operations are performed. Division and multiplication are performed before addition or subtraction. There is a common tendency to expect calculations to be done from left to right, as the equation would be read in English. Calc evaluates the entire formula. Then, based upon programming precedence, Calc breaks the formula down, carrying out operations in the order described in Table 4, executing:

1) exponentiation (raising to a power)
2) multiplication and division
3) addition and subtraction

For this reason, when creating formulas, test them to ensure you get the correct result. Following is an example of the order of calculation in operation.

Table 4 – Order of Calculation

Left To Right Calculation	***Ordered Calculation***
1+3*2+3 = 11 1+3=4, then 4 X 2 = 8, then 8 + 3 = 11	=1+3*2+3 result 10 3*2=6, then 1 + 6 + 3 = 10
Another possible intention could be: 1+3*2+3 = 4 * 5 = 20	The program resolves the multiplication of 3 X 2 before dealing with the numbers being added.

If you intend the result to be either of the two possible solutions on the left, the way to achieve these results would be to order the formula as:

((1+3) * 2)+3 = 11 (1+3) * (2+3) = 20

Note	Use parentheses to group operations in the order you intend; for example, `=B4+G12*C4/M12` might become `=((B4+G12)*C4)/M12`.

Calculations Linking Sheets

Another powerful feature of Calc is its ability to link data through several worksheets. The names of worksheets help identify where specific data may be found. For example, a name such as Payroll or Boise Sales is much more meaningful than Sheet1.

Note	The function SHEET() returns the sheet number in the collection of spreadsheets. There are several worksheets in each book; numbered from the left: Sheet1, Sheet2, and so forth. If you drag the worksheets to different locations among the tabs, SHEET() returns the number referring to the current position of this worksheet.

Consider a business setting in which revenues and costs of each of its branch operations are combined into a single worksheet.

	A	K	L	M	N
2	Flowing Abundantly, Ltd.				
3	Combined Sales YTD				
4					
5					
6		Oct	Nov	Dec	YTD
7	Revenue:				
8	Greenery Sales	36,288	52,874	81,335	**1,283,107**
9	Fertilizer Sales	16,822	3,825	3,600	**697,634**
10	Earth Sales	2,019	459	432	**84,479**
11	Sub-Total	**55,129**	**57,158**	**85,367**	**2,065,220**
12					
13	Cost of Sales:				
14	Wholesaler Purchases	18,744	19,434	29,025	**702,175**
15	Sales Tax	6,064	6,287	9,390	**227,174**
16	Sub-Total	**24,808**	**25,721**	**38,415**	**929,349**
17					
18	Total Revenue:	**30,321**	**31,437**	**46,952**	**1,135,871**
19					
20	Expenses:				

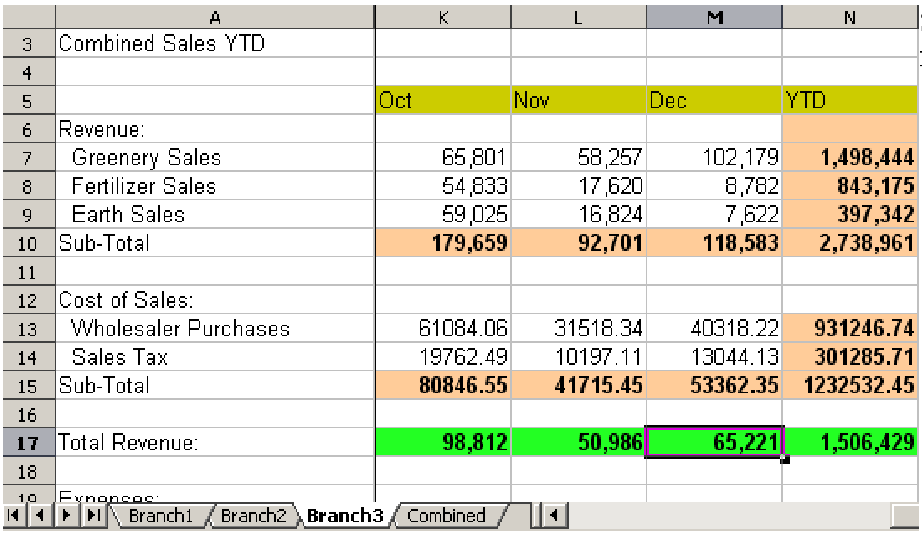
Branch1 Branch2 Branch3 Combined

Sheet containing data for Branch 1.

	A	K	L	M	N
2	Flowing Abundantly, Ltd.				
3	Combined Sales YTD				
4					
5					
6		Oct	Nov	Dec	YTD
7	Revenue:				
8	Greenery Sales	38,251	14,899	49,588	**1,027,538**
9	Fertilizer Sales	6,120	2,384	7,934	**164,406**
10	Earth Sales	734	286	952	**19,729**
11	Sub-Total	**45,106**	**17,569**	**58,474**	**1,211,673**
12					
13	Cost of Sales:				
14	Wholesaler Purchases	15,336	5,973	19,881	**411,969**
15	Sales Tax	4,962	1,933	6,432	**133,284**
16	Sub-Total	**20,298**	**7,906**	**26,313**	**545,253**
17					
18	Total Revenue:	**24,808**	**9,663**	**32,161**	**666,420**
19					
20	Expenses:				

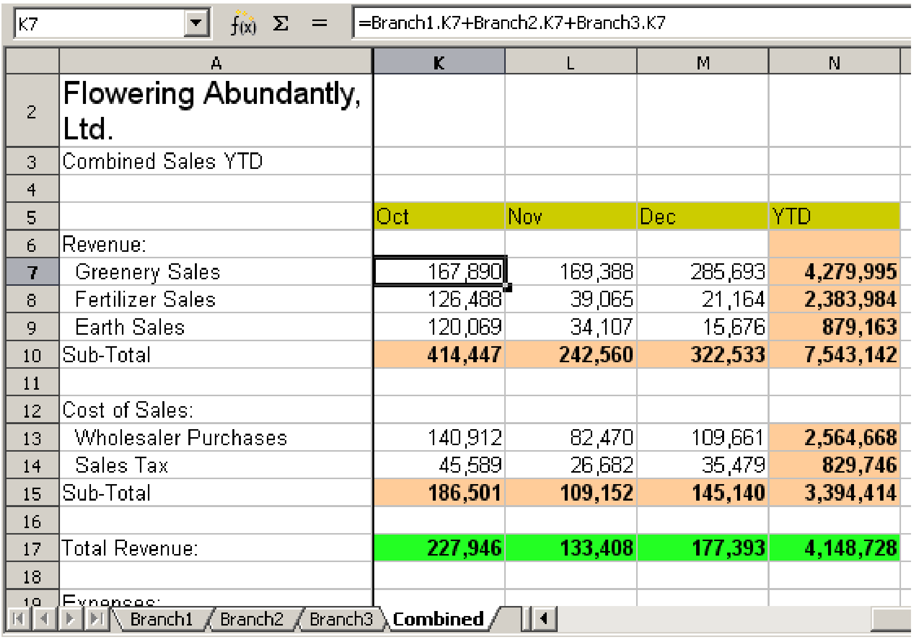
Branch1 **Branch2** Branch3 Combined

Sheet containing data for Branch 2.



	A	K	L	M	N
3	Combined Sales YTD				
4					
5		Oct	Nov	Dec	YTD
6	Revenue:				
7	Greenery Sales	65,801	58,257	102,179	**1,498,444**
8	Fertilizer Sales	54,833	17,620	8,782	**843,175**
9	Earth Sales	59,025	16,824	7,622	**397,342**
10	Sub-Total	**179,659**	**92,701**	**118,583**	**2,738,961**
11					
12	Cost of Sales:				
13	Wholesaler Purchases	61084.06	31518.34	40318.22	**931246.74**
14	Sales Tax	19762.49	10197.11	13044.13	**301285.71**
15	Sub-Total	**80846.55**	**41715.45**	**53362.35**	**1232532.45**
16					
17	Total Revenue:	**98,812**	**50,986**	**65,221**	**1,506,429**
18					
19	Expenses:				

Sheet containing data for Branch 3.



	A	K	L	M	N
2	Flowering Abundantly, Ltd.				
3	Combined Sales YTD				
4					
5		Oct	Nov	Dec	YTD
6	Revenue:				
7	Greenery Sales	167,890	169,388	285,693	**4,279,995**
8	Fertilizer Sales	126,488	39,065	21,164	**2,383,984**
9	Earth Sales	120,069	34,107	15,676	**879,163**
10	Sub-Total	**414,447**	**242,560**	**322,533**	**7,543,142**
11					
12	Cost of Sales:				
13	Wholesaler Purchases	140,912	82,470	109,661	**2,564,668**
14	Sales Tax	45,589	26,682	35,479	**829,746**
15	Sub-Total	**186,501**	**109,152**	**145,140**	**3,394,414**
16					
17	Total Revenue:	**227,946**	**133,408**	**177,393**	**4,148,728**
18					
19	Expenses:				

Sheet containing combined data for all branches.

Figure 10: Combining data from several sheets into a single sheet

These hypothetical spreadsheets have been configured to have identical structures by setting up the first Branch spreadsheet in preparation for:

- inputting data
- formatting cells
- preparing the formulas for the various sums of rows and columns

1) On the worksheet tab, right-click and select **Rename Sheet**. Type Branch1. Right-click on the tab again and select **Move/Copy Sheet.**
2) In the Move/Copy Sheet dialog, select the *Copy* option and select Sheet 2 in the area *Insert before*. Click **OK**, right-click on the tab of the sheet Branch1_2, and rename it to Branch2. Repeat to produce the Branch3 and Combined worksheets.

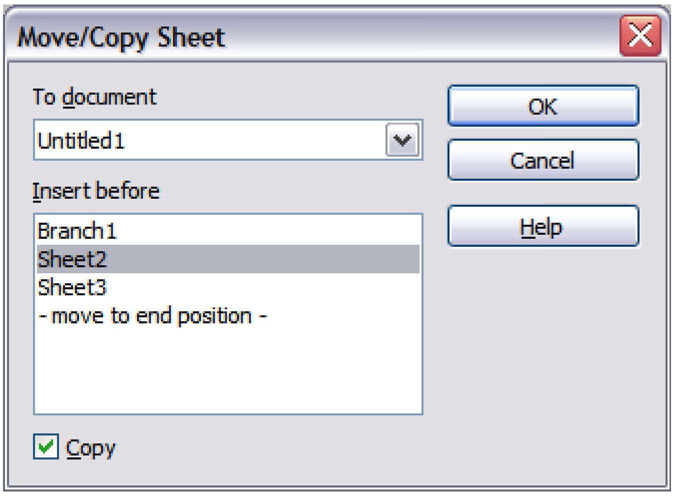


Figure 11: Copying a worksheet

3) Enter the data for Branch 2 and Branch 3 into the respective sheets. Each sheet stands alone and reports the results for the individual branches.
4) In the Combined worksheet, click on cell K7. Type =, click on the tab Branch1, click on cell K7, press +, repeat for sheets Branch2 and Branch3, and press Enter. You now have a formula in cell K1 that adds the revenue from Greenery Sales for the 3 Branches.

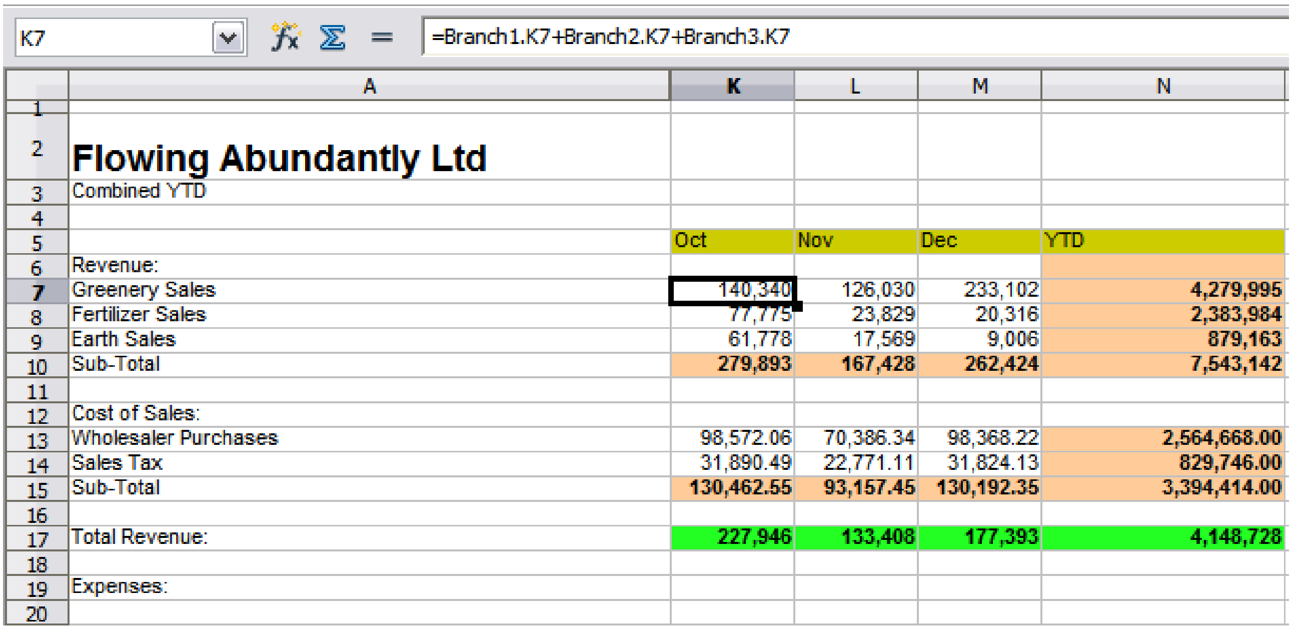


Figure 12: Combined worksheet showing linking between branch sheets

5) Copy the formula, highlight the range K7..N17, click **Edit > Paste Special**, uncheck the Paste all and Formats boxes in the Selection area of the dialog box, and click OK. You will see the following message:

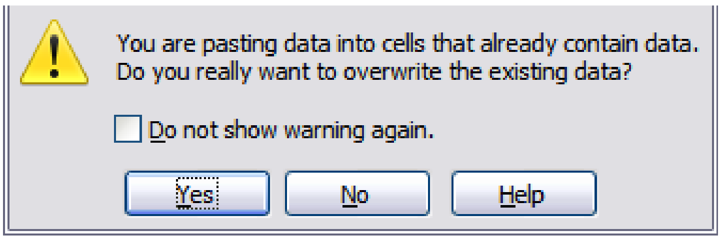


Figure 13: Linking sheets: pasting a formula to a cell range

6) Click **Yes**. You have now copied the formulas into each cell while maintaining the format you set up in the original worksheet. Of course, in this example, you would have to tidy the worksheet up by removing the zeros in the non-formatted rows.

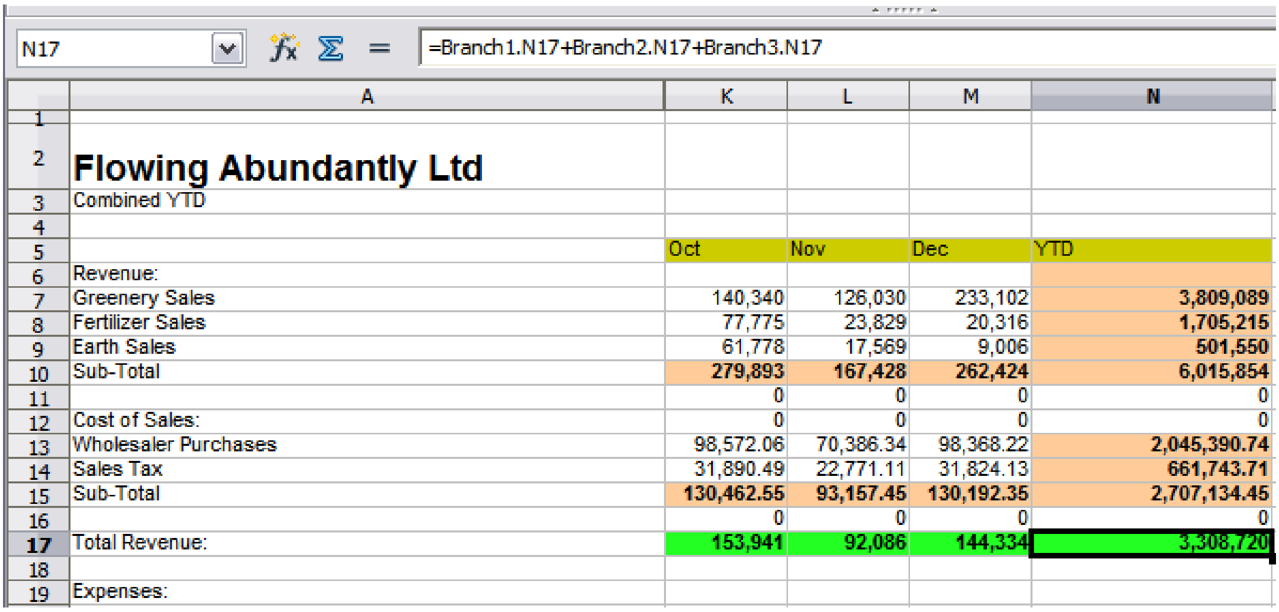


Figure 14: Linking Sheets: Copy Paste Special from K7...N17

Note OpenOffice's default is to paste all the attributes of the original cell(s) - formats, notes, objects, text strings, and numbers.

The Function Wizard can also be used to accomplish such linking. The use of this Wizard is described in detail in the section on Functions.

Understanding Functions

Calc includes over 350 functions that help you analyze and reference data. Many of these functions are for use with numbers, but others can be used with dates and times or even text. A function may be as simple as adding two numbers together or finding the average of a list of numbers. Alternatively, it may be as complex as calculating the standard deviation of a sample or a hyperbolic tangent of a number.

Typically, the name of a function is an abbreviated description of what the function does. For instance, the FV function gives the future value of an investment, while BIN2HEX converts a binary number to a hexadecimal number. By tradition, functions are entered entirely in upper case letters, although Calc will read them correctly if they are in lower or mixed case.

A few basic functions are somewhat similar to operators. Examples:

+ This operator adds two numbers together for a result. SUM() adds individual cells or cell ranges together.

* This operator multiplies two numbers together for a result. PRODUCT() does the same for multiplying that SUM() does for adding.

Each function has a number of *arguments* used in the calculations. These arguments may or may not have their own name. Your task is to enter the arguments needed to run the function. In some cases, the arguments have predefined choices, and you may need to refer to the online help or Appendix B (Description of Functions) in this book to understand them. More often, however, an argument is a value you enter manually or one already entered in a cell or range of cells on the spreadsheet. In other words, an argument is what the function must work on. In Calc, you can enter values from other cells by typing their name or range or by selecting cells with the mouse. If the values in the cells change, the result of the function is automatically updated.

For compatibility, functions and their arguments in Calc have almost identical names to their counterparts in Microsoft Excel. However, both Excel and Calc have functions that the other lacks. Occasionally, functions with the same names in Calc and Excel have different arguments or slightly different names for the same argument—neither of which can be imported to the other. However, most functions can be used in both Calc and Excel without any change.

Understanding the Structure of Functions

All functions have a similar structure, which is worth knowing to be able to troubleshoot.

To give a typical example, the structure of a function to count cells that match entered search criteria is:

```
= DCOUNT (Database;Database field;Search_criteria)
```

Since a function cannot exist independently, it must always be part of a formula. Consequently, even if the function represents the entire formula, there must be an = sign at the start of the formula. Regardless of where in the formula a function is, the function will start with its name, such as DCOUNT, in the example above. After the name of the function comes its arguments; all arguments are required unless specifically listed as optional.

Arguments are added within the parentheses and are separated by semicolons.

Note	OpenOffice uses the semicolon as an argument list separator, unlike Excel, which uses a comma. OpenOffice uses the semicolon because the comma is the decimal mark in numbers in many locales.

Many arguments are a number. A Calc function can take up to thirty numbers as an argument. That may not sound like much at first. However, when you realize that the number can be not only a number or a single cell but also an array or range of cells containing several or even hundreds of cells, the apparent limitation vanishes.

Depending on the nature of the function, arguments may be entered as follows:

"text data"	The quotes indicate text or string data is being entered.
9	The number nine is being entered as a number.
"9"	The character nine is being entered as text
A1	The address for whatever is in Cell A1 is being entered

Nested Functions

Functions can also be used as arguments within other functions. These are called nested functions.

```
=SUM(2;PRODUCT(5;7))
```

Imagine that you are designing a self-directed learning module. During the module, students do three quizzes and enter the results in cells A1, A2, and A3. In A4, you can create a nested formula that begins by averaging the results of the quizzes with the formula `=AVERAGE(A1:A3)`. The formula then uses the `IF` function to give the student feedback that depends upon the average grade on the quizzes. The entire formula would read:

```
=IF(AVERAGE(A1:A3) >85; "Congratulations! You are ready to advance to
the next module"; "Failed. Please review the material again. If
necessary, contact your instructor for help")
```

Depending on the average, the student would receive the message for either congratulations or failure.

Notice that the nested formula for the average does not require its own equal sign. The one at the start of the equation is enough for both formulas.

We've used simple examples to explain the concept more clearly, but a Calc formula can quickly become complex through nesting of functions. You should be careful not to overuse nesting; it can be difficult to find errors in a heavily nested function. Particularly while developing a calculation, it is often easier to place parts of the calculation in different cells and bring the final calculation together through cell references. That makes it easier to see the source of an unexpected result.

Note	Calc keeps the syntax of a formula displayed in a tool-tip next to the cell as a handy memory aid as you type.

Remembering the name and syntax of each function can be difficult. Calc provides a *Function List* panel (Figure 15) in the Sidebar that organizes and briefly describes the available functions. The panel is selected by clicking the icon in the Sidebar.

The Function List includes a brief description of each function and its arguments; highlight the function and look at the bottom of the pane to see the description. If necessary, hover the cursor over the division between the list and the description; when the cursor becomes a two-headed arrow, drag it upwards to increase the space for the description. Double-click on a function's name to add it to the current cell, together with placeholders for each of the function's arguments. In the example shown, the MAX function has been inserted, and its argument is shown as *number*. Of course, you would not want to enter just one number, and the function description section of the Sidebar shows that multiple numbers can be entered, separated by semicolons. Rather than a plain number, you would usually enter one or more cell ranges containing numbers.

Using the Function List is almost as fast as manual entry and has the advantage of not requiring you to memorize a function you want to use. In theory, it should also be less error-prone. In practice, some users may fumble when replacing the placeholders with values. Another feature is the ability to display the last functions used. You can see in Figure 15 the drop-down list set to *Last Used*. This same list can be used to display categories of functions such as *Mathematical, Date & Time,* and

Financial, among others, which can be very helpful in finding a function among the many available options.

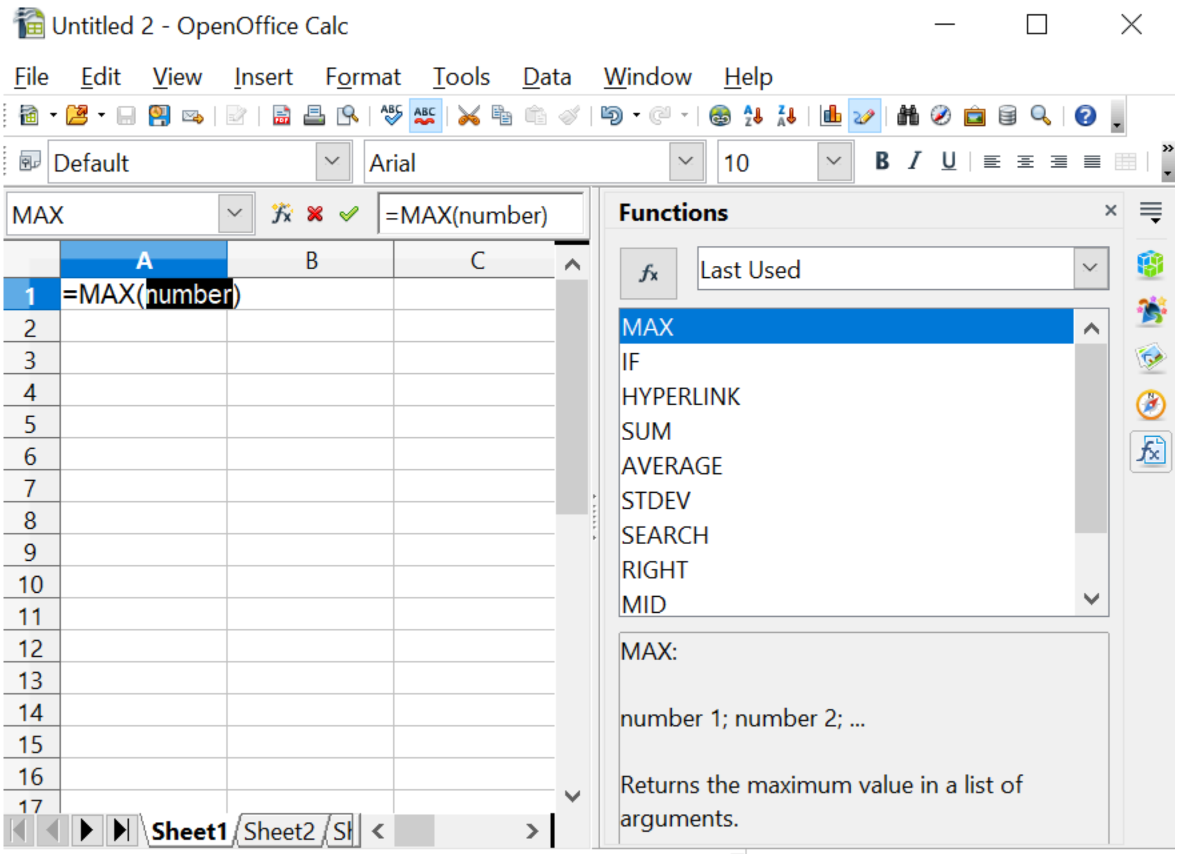


Figure 15: Function List docked to right side of Calc window

Function Wizard

A more elaborate tool for inputting functions is available in the *Function Wizard* (Figure 17). To open the Function Wizard, either choose **Insert > Function** or click the button on the Function toolbar or press *Ctrl+F2*. Once open, the Function Wizard provides the same help features as the Function List but adds fields in which you can see the result of a completed function, and the result of any larger formula of which it is a part.

Select a category of functions to shorten the list. Then scroll down through the named functions and select the required one. When you select a function, its description appears on the right-hand side of the dialog. Double-click on the required function. Figure 17 shows the Function Wizard after double-clicking on the SUM entry in the *Function* list.

The Wizard now displays an area to the right where you can enter data manually in text boxes or click the Shrink button to shrink the wizard so you can select cells from the worksheet. Figure 16 shows the Function Wizard after shrinking.

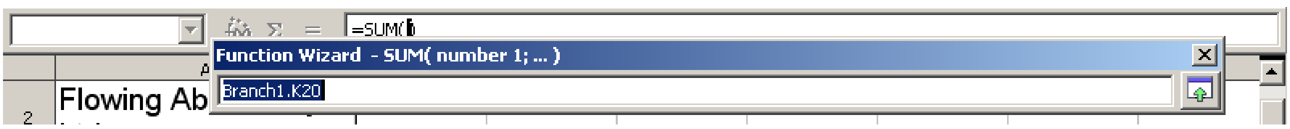


Figure 16: Function Wizard after shrinking

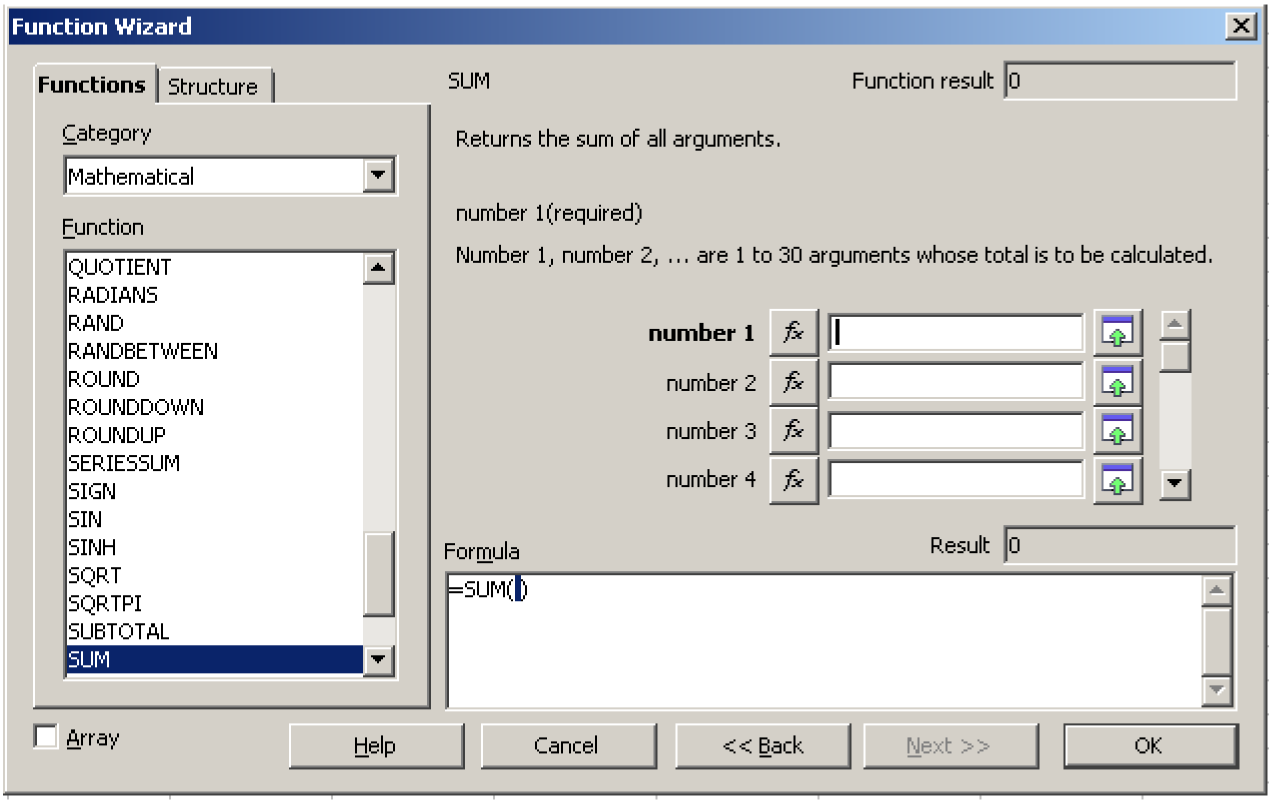


Figure 17: Functions tab of Function Wizard.

To select cells, click the cell directly or hold down the left mouse button and drag to select the required area.

When the area has been selected, click the Shrink button again to return to the wizard.

If multiple arguments are needed, select the next text box below the first, and repeat the selection process for the next cell or range of cells. Repeat this process as often as required. The Wizard will accept up to 30 ranges or arguments in the SUM function.

Click **OK** to accept the function and add it to the cell to get the result.

You can also select the *Structure* tab (Figure 18) to see a tree view of the parts of the formula. The main advantage over the Function List is that each argument is entered in its own field, making it easier to manage. The price of this reliability is slower input. That's often a small price to pay since precision is generally more important than speed when creating a spreadsheet.

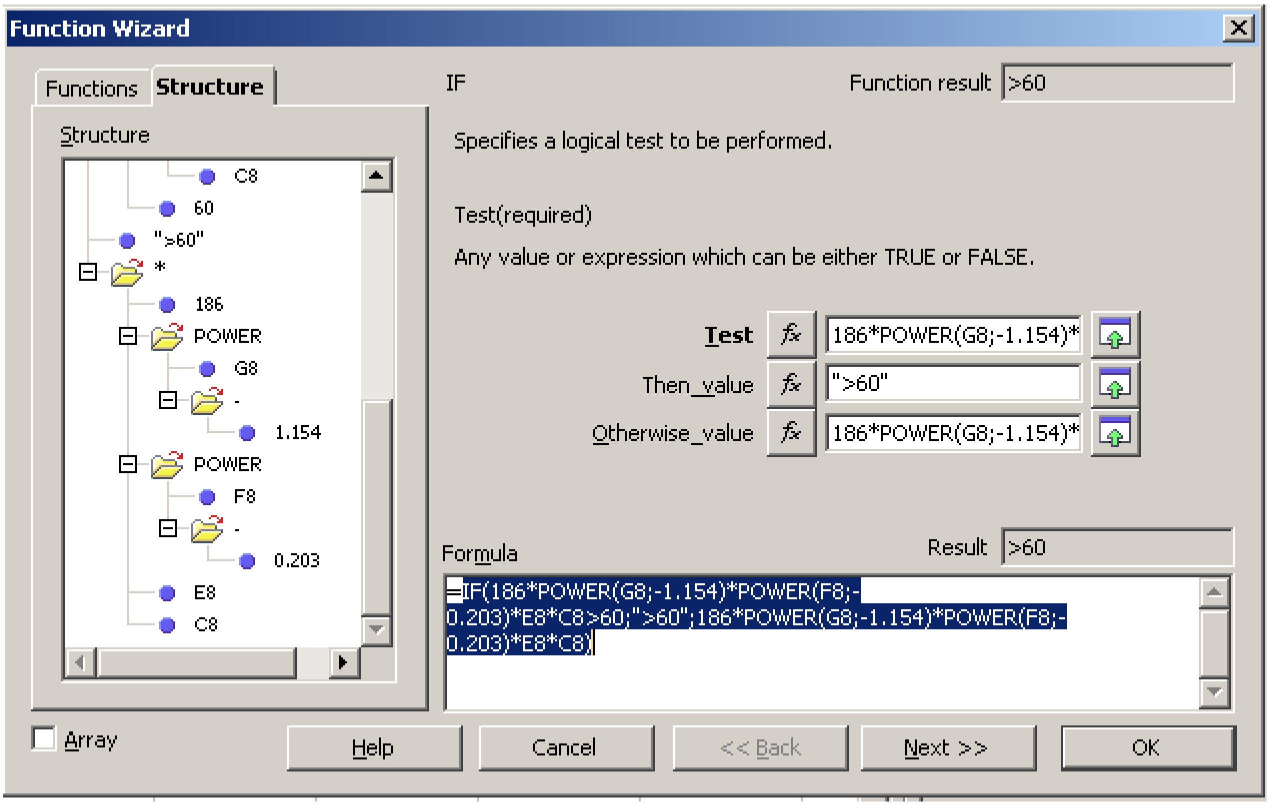
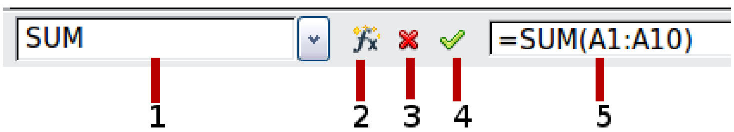


Figure 18: Structure tab of Function Wizard

Entering Formulas in Cells or the Input Line

Formulas can also be entered directly into cells or the Input line. Start the entry with an equal sign and type the required numbers, text, and functions. Calc will provide tips for the function arguments. You can use the mouse to select cell ranges via click & drag. After you enter a function on the Input line, press the *Enter* key or click the **Accept** button on the Function toolbar to add the function to the cell and get its result.



1 Name Box showing list of common functions
2 Function Wizard
3 Cancel
4 Accept
5 Input Line

Figure 19: The Function toolbar

If you see the formula in the cell instead of the result, then *Formulas* are selected for display in **Tools > Options > OpenOffice Calc > View > Display**. Deselect *Formulas*, and the result will display. However, you can still see the formula in the Input line.

Strategies for Creating Formulas and Functions

You can take several broad approaches when creating a formula. In deciding which approach to take, consider how many other people will need to use the worksheets, the life of the worksheets, and the variations that could be encountered in use of the formula.

If people other than yourself will use the spreadsheet, make sure that it is easy to see what input is required, and where. Explanation of the purpose of the spreadsheet, basis of calculation, input required, and output(s) generated are often placed on the first worksheet.

A spreadsheet you build today, with many complicated formulas, may not be quite so obvious in its function and operation in 6 or 12 months. Use comments and notes liberally to document your work.

You might be aware that you cannot use negative values or zero values for a particular argument, but if someone else inputs such a value, will your formula be robust or simply return a standard (and often not too helpful) Err: message? It is a good idea to trap errors using some form of logic statements or conditional formatting.

Place a Unique Formula in Each Cell

The simplest approach is to write formulas for complete calculations in whichever cells are convenient. This is often a sufficient strategy for simple calculations. For example, a personal budget might have columns for the date, transaction type, and amount, with income as positive numbers and costs as negative numbers. A SUM() formula could then return the change in the account balance. Though this approach would probably not suffice for even a tiny business, it will work if you just want to track at a high level how closely your expenses match your income. There are many cases where the input data and the questions to be answered are uncomplicated, and there is no need for a more elaborate spreadsheet. The disadvantage of doing complete calculations in one cell is that it rapidly becomes difficult to understand and troubleshoot formulas as they become more complicated.

Break Formulas into Parts and Combine the Parts

The second strategy is similar to the first, but instead, you break down longer formulas into smaller parts and then combine the parts into the whole. Many examples of this type exist in complex scientific and engineering calculations, where interim results are used in several places in the worksheet. Whenever you start nesting functions, that is, using functions within functions, you should consider breaking the calculation into parts.

A very simple example is shown in Figure 20. The price of a product is calculated based on the color, minimum temperature, and square meters required. Three quantities are multiplied to get the final answer. The left side of the figure shows a successful calculation using an "all-in-one" method in cell A9 and a combination of sub–formulas in A16. The formulas used in each cell of A9, A12:A14, and A16 are shown in the neighboring column B cells. You would not typically include the text in column B in a spreadsheet. It is included here to make the figure easier to understand.

Don't worry if you don't understand what each formula does. The important point is that cells A12:A14 calculate parts of the larger formula in A9 and these parts are combined in A16. An advantage of combining sub–formulas can be seen on the right side of the figure. A negative number has been entered in cell A2, and this causes the SQRT() function to return the error message Err:502. This causes the all-in-one formula to also return that error message, but it is unclear what part of the formula causes the error. The combined result in A16 also returns the error, but looking at the sub–formulas in A12:A14, it is clear that the SQRT() function is the culprit.

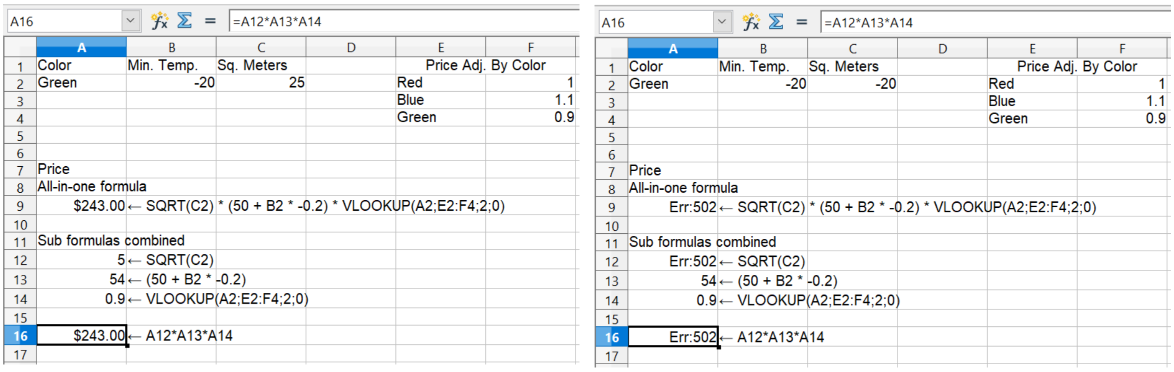


Figure 20: Calculation and errors with all-in-one or combined formulas

Breaking out parts of the calculation helps in tracking errors and unexpectedly large or small results. It is generally much easier to troubleshoot a calculation that uses several quantities if the individual values can be easily seen. This requires using more cells, but it can save a lot of time.

Use the Basic Editor to Create Functions

A third strategy is to use the Basic editor to create your own functions and macros. This approach can be used where the result would greatly simplify the use of the spreadsheet and keep the formulas simple, with a better chance of avoiding errors. This approach can also make maintenance easier by keeping corrections or updates in one central location. The use of macros is described in Chapter 12 of this book and is a specialized topic. The danger of overusing macros and custom functions is that the principles upon which the spreadsheet is based become much more difficult for users other than the original author to see.

Finding and Fixing Errors

It is common to find situations where errors are displayed. Even with all the tools Calc has that help you enter formulas, making mistakes is easy. Many people find inputting numbers difficult, and may make a mistake about the kind of entry a function's argument needs. In addition to correcting errors, you may want to find the cells used in a formula to change their values or to check the answer.

Calc provides three tools for investigating formulas and the cells that they reference: error messages, color coding, and the Detective.

Error Messages

Error messages are Calc's most basic tool for locating and correcting errors. Error messages display in a formula’s cell or in the Function Wizard instead of the result.

An error message for a formula is usually a three-digit number from 501 to 527 or sometimes unhelpful text such as #NAME?, #REF, or #VALUE. The error number appears in the cell, and a brief explanation of the error is displayed on the right side of the status bar.

Most error messages indicate a problem with how the formula was input, although several indicate that you have run up against a limitation of either Calc or its current settings.

Error messages are not at all user-friendly and may intimidate some users. However, they are valuable clues to correcting mistakes. You can find detailed explanations of them by searching for *error codes; list of* in the OpenOffice Calc help file. A few of the most common are shown in the following table.

#NAME? (525)	No valid reference exists for the argument.
#REF (525)	The column, row, or sheet for the referenced cell is missing.
#VALUE (519)	The value for one of the arguments is not the type that the argument requires. The value may be entered incorrectly; for example, double quotation marks may be missing around the value. At other times, a cell or range may have the wrong kind of value, such as text instead of numbers.
509	An operator such as an equals sign is missing from the formula.
510	An argument is missing from the formula.
502	Function argument is not valid, for example, a negative number for the root function.

Examples of Common Errors

Err:532 #DIV/0!

This error results from dividing a number by either the number zero (0) or a blank cell. There is an easy way to avoid this problem. When you have a zero or blank cell displayed, use a conditional function. Figure 21 shows the result of dividing column A by column B when some cells in B contain zero or are blank.

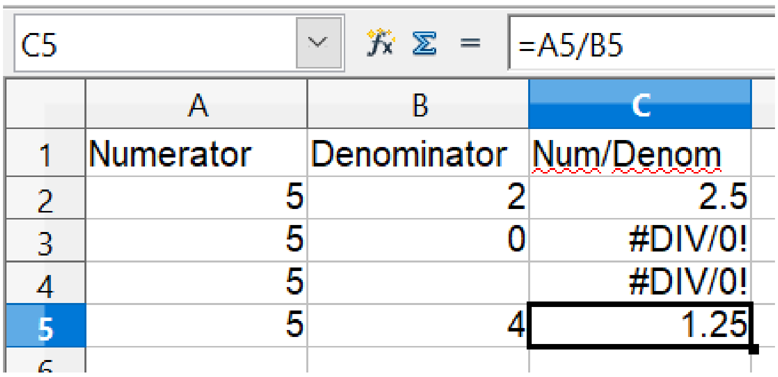


Figure 21: Examples of #DIV/0!, Division by zero

It's common to find errors like this in situations in which data was not reported or was reported incorrectly. When such an error is possible, an IF function can be used to display the data correctly. The formula

```
=IF(B2 <>0;A2/B2;"Zero Denom.")
```

can be entered. The formula is then copied over the remainder of Column C. This formula roughly means: “If B2 is not 0, then compute A2 divided by B2; otherwise, display ‘Zero Denom.’”. Note that the operator **<>** means **not equal to**.

It is also possible for the last parameter to use empty double quotes for a blank to be entered or a different formula with a standardized number being substituted for the lower number.

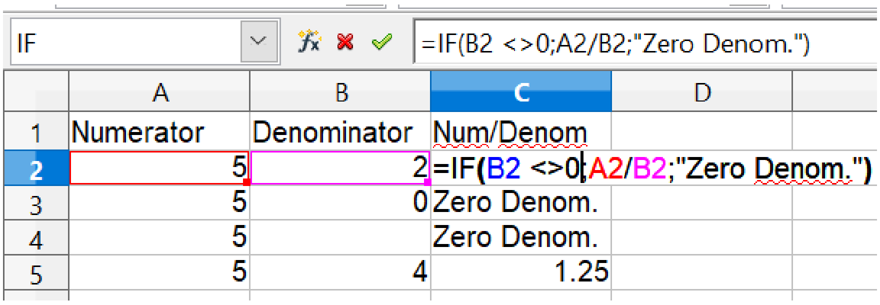


Figure 22: Division by zero solution

#VALUE Non-existent Value and #REF! Incorrect References

The non-existent value error is also very common, appearing frequently when a user copies a formula over a selected area. When copying, it is typical for the program to increment the represented cells. If you were copying downward from cell B3, the program would automatically substitute cell B4 into the next lower cell, and so on, until the end of the copying process. If that next cell contains text or a value inappropriate for the formula, this error may result. The difficulty often occurs when one or more of the parameters in the formula need to be fixed.

Note	If you need to keep a reference to B3 fixed while copying a formula, you can give the cell B3 a name such as TotalExpenses. In that way, the program will carry that name to each succeeding formula being copied. Alternatively, use an absolute reference, B3.

Color Coding for Input

Another useful tool when reviewing a formula is color coding for input. When you select a formula that has already been entered by double-clicking in the cell or editing the formula in the Input line, the cells or ranges used for each argument in the formula are outlined in color. This can be very helpful in confirming that the formula references the intended cells.

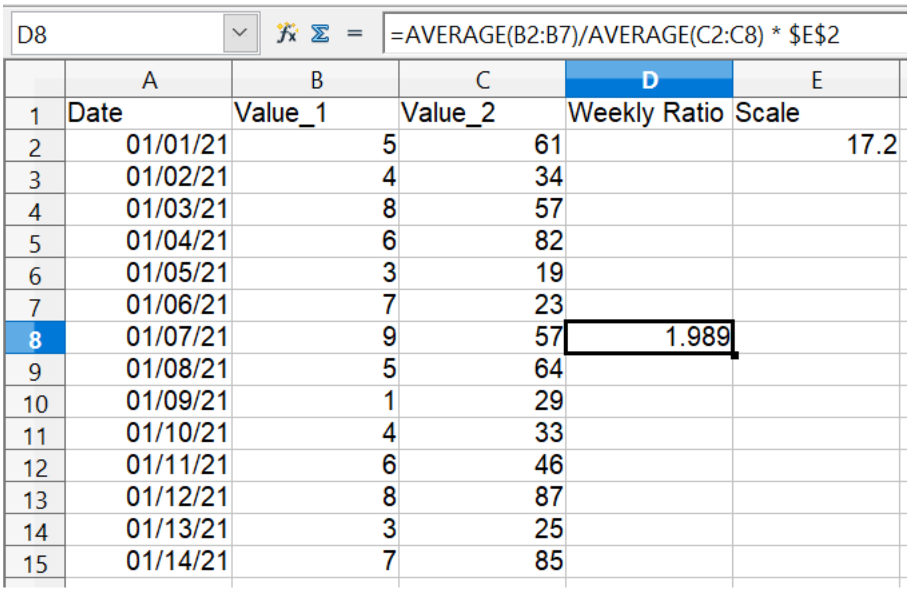
Calc uses eight colors for outlining referenced cells, starting with blue for the first cell and continuing with red, magenta, green, dark blue, brown, purple, and yellow before cycling through the sequence again.

The Detective

In a long or complicated spreadsheet, color coding becomes less useful. In these cases, consider using the submenu under **Tools > Detective**. The Detective is a tool for checking which cells are used as arguments by a formula (precedents) and which other formulas it appears in (dependents). It can also be used for tracing errors, flagging invalid data (information in cells that is not proper for a function’s argument), or even removing precedents and dependents.

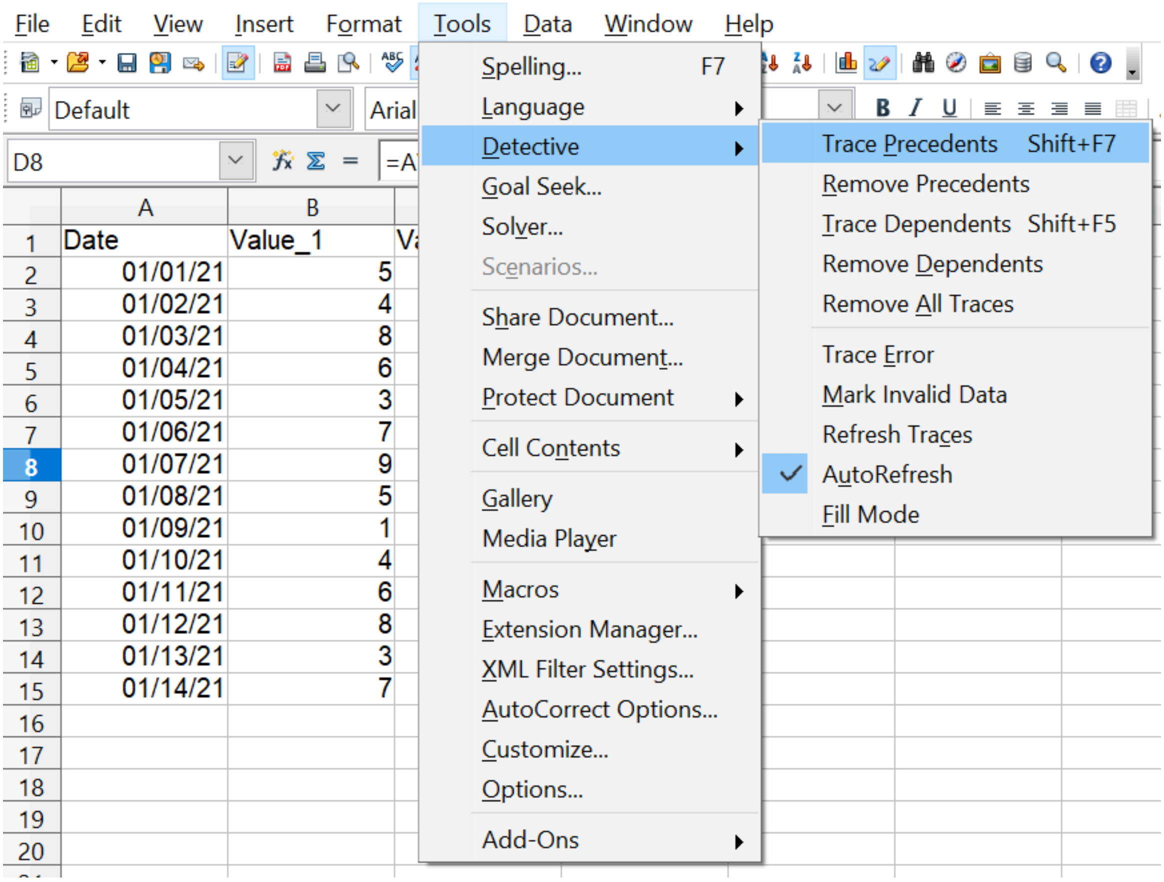
To use the Detective, select a cell with a formula, then start the Detective. The spreadsheet shows lines ending in circles to indicate precedents and lines ending in arrows for dependents. The lines show the flow of information.

Use the Detective to assist in following the precedents referred to in a formula in a cell. By tracing these precedents, you can frequently find the source of the errors. Place the cursor in the cell in question and then choose **Tools > Detective > Trace Precedents** from the menu bar or press *Shift+F7*. Figure 23 shows a simple example of tracing precedents.

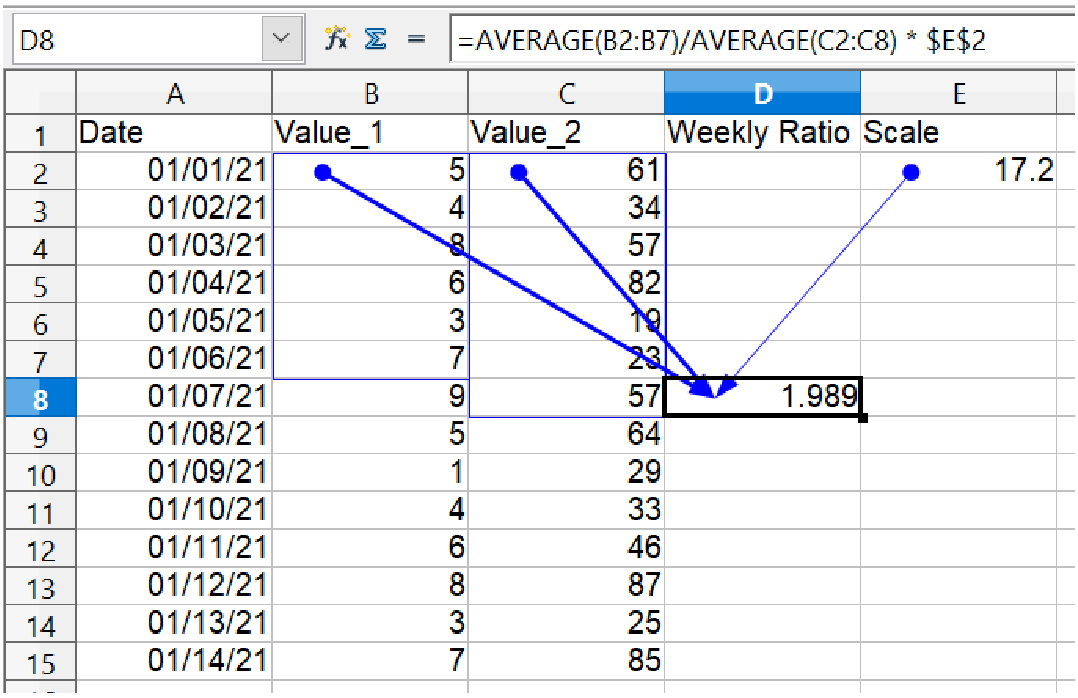


Cursor placed in cell

Figure 23: Tracing precedents using the Detective



a) Initiate trace by clicking Trace Precedents



b) Source area highlighted in Blue, with arrows pointing to the calculation cell

(continued): Tracing precedents using the Detective

We are concerned that the number shown in Cell D8 is incorrectly stated. The cause can be seen in the highlighted cells. The ranges spanned in columns B and C are different, with column B missing the last day of the week.

In other cases, we must trace the error. Use the Trace Error function under **Tools > Detective > Trace Error** to find the cells that cause the error.

Examples of Functions

For novices, functions are among the most intimidating features of OpenOffice's Calc. New users quickly learn that functions are an important feature of spreadsheets, but there are almost four hundred, and many require input that assumes specialized knowledge. Fortunately, Calc includes dozens of functions that anyone can use.

Basic Arithmetic and Statistic Functions

The most basic functions create formulas for basic arithmetic or for evaluating numbers in a range of cells.

Basic Arithmetic

The simple arithmetic functions are addition, subtraction, multiplication, and division.

Except for subtraction, each of these operations has its own function:

- SUM for addition
- PRODUCT for multiplication
- QUOTIENT for integer division. This may not do what you expect, as explained below.

Traditionally, subtraction does not have a function.

SUM and PRODUCT are useful for entering ranges of cells in the same way as any other function, with arguments in parentheses after the function name.

However, for basic equations using only a few values, many users prefer the time-honored computer symbols for these operations, using the plus sign (+) for addition, the hyphen (–) for subtraction, the asterisk (*) for multiplication, and the forward-slash (/) for division. These symbols are quick to enter and don't require your hands straying from the keyboard.

A similar choice is also available if you want to raise a number by the power of another. Instead of entering =POWER(A1;2), you can enter =A1^2.

Moreover, arithmetic operators have this advantage. You enter them in a way and an order that closely approximates human–readable format, more so than the spreadsheet-readable format used by the equivalent function. For instance, instead of entering =SUM (A1:A2), or possibly =SUM (A1;A2), you enter =A1+A2. This almost-human-readable format is especially useful for compound operations, where writing =A1*(A2+A3) is briefer and easier to read than =PRODUCT(A1;SUM(A2:A3)).

The main disadvantage of using arithmetical operators is that you cannot directly use a range of cells. In other words, to enter the equivalent of =SUM (A1:A5), you would need to type =A1+A2+A3+A4+A5. This rapidly becomes impractical.

Otherwise, whether you use a function or an operator is mainly up to you. However, suppose you use spreadsheets regularly in a group setting such as a class or an office. In that case, you might want to standardize on an entry format so that everyone who handles a spreadsheet becomes accustomed to a standard input.

The QUOTIENT function returns only the integer part of the division. For example, QUOTIENT(18; 5) returns 3, not 3.6. Also, the function does not accept cell ranges since sequentially doing integer division is not useful. Normal division is done with the / operator.

Simple Statistics

Another common use for spreadsheet functions is to extract summary information from a list, such as a series of test scores in a class or a summary of earnings per quarter for a company.

You can scan a list of figures if you want basic information, such as the highest or lowest entry or the average. The only trouble is that the longer the list, the more time you'll spend, and the more likely you will miss what you're looking for. Instead, it is usually quicker and more efficient to enter a function. Such reasons explain the existence of a function like COUNT, which does no more than give the total number of numeric entries in the designated cell range.

Similarly, to find the highest or lowest entry, you can use MIN or MAX. For each of these formulas, all arguments are either a range of cells or a series of cells entered individually.

Each also has a related function, MINA or MAXA, which performs the same function but treats a cell formatted for text as having a value of 0 (The same treatment of text occurs in any variation of another function that adds an "A" to the end). For example, if you have four cells containing 5, 7, 3, and Z, the MIN function will return 3, ignoring the text Z, and the MINA function will return 0, treating the text Z as a zero.

For more flexibility in similar operations, you could use LARGE or SMALL, both of which add a specialized argument of rank. If the rank is 1 with LARGE, you get the same result as with MAX. However, if the rank is 2, then the result is the second largest result. Similarly, a rank of 2 with SMALL returns the second–smallest number.

LARGE and SMALL are handy as permanent controls since you can quickly scan multiple results by changing the rank argument.

You would need to be an expert to find the Poisson Distribution (a mathematical model of the number of outcomes obtained in a suitable interval of time and space) of a sample or to find the skew or negative binomial of a distribution. (If you are, you'll find functions in Calc for such things.). However, for the rest of us, there are simpler statistical functions that you can quickly learn to use.

In particular, if you need an average, you have more than one to choose from. You can find the arithmetical means—that is, the result when you add all entries in a list and then divide by the number of entries by entering a range of numbers when using AVERAGE or AVERAGEA to include text entries and to give them a value of zero.

In addition, you can get other information about the data set:

- MEDIAN: The number for which half the values in the list are higher and half are lower.
- MODE: The most common entry in a list of numbers.
- QUARTILE: The entry at a set position in the array of numbers. Besides the cell range, you enter the type of Quartile: 0 for the lowest entry, 1 for the value of 25%, 2 for the value of 50%, 3 for 75%, and 4 for the highest entry. Note that the result for types 1 through 3 may not represent an actual item entered.
- RANK: The sorted position of a given entry in the entire list, in ascending or descending order. You need to enter the value for the entry, the range of entries, and the type of rank (**0** for the rank in descending order or **1** for the rank in ascending order). For example, if A1:A4 contain 5, 7, 8, 2, RANK(5, A1:A4;0) returns 3 because 5 is the third element if the values are sorted in descending order.

Some of these functions overlap; for example, MIN and MAX are both covered by QUARTILE. In other cases, a custom sort or filter might give much the same result. Which you use depends on your preferences and your needs. Some might prefer to use MIN and MAX because they are easy to remember, while others might prefer QUARTILE because it is more versatile.

Rounding off Numbers

For statistical and mathematical purposes, Calc includes a variety of ways to round off numbers. You may also be familiar with some of these methods if you’re a programmer. However, you don’t need to be a specialist to find some of them helpful. You may want to round off for billing purposes or because decimal places don’t translate well into the physical world—for instance, if the parts you need come in packages of 100, then the fact you only need 66 becomes irrelevant. You need to round up for ordering. By learning the options for rounding up or down, you can make your spreadsheets more immediately useful.

When you use a rounding function, you have two choices about how to set up your formulas. If you choose, you can nest a calculation within one of the rounding functions. For instance, the formula =ROUND((SUM(A1;A2)) adds the figures in cells A1 and A2, then rounds the result off to the nearest whole number. However, even though you don’t need to work with exact figures daily, you may still want to refer to them occasionally. If that is the case, you are probably better off separating the two functions, placing =SUM(A1;A2) in cell A3 and =ROUND (A3) in A4, and clearly labeling each function.

Rounding Methods

Some examples of all of the functions mentioned in the following paragraphs are shown below in Table 5. The table includes some comments highlighting the functions' behavior with negative numbers since the results may not be what you expect.

ROUND

Syntax: `ROUND(Number; Count)`

The most basic function for rounding numbers in Calc is ROUND. If used without the optional Count argument, this function will round off a number according to the usual rules of symmetric arithmetic rounding: a decimal place of .4 or less gets rounded down, while one of .5 or more gets rounded up. Using the *Count* argument, you can specify the number of decimal places to include. For instance, if *Count* was set to 1, 48.65 would round to 48.7. Negative values of *Count* cause rounding to multiples of positive powers of 10. Rounding 48.65 with *Count* set to -1 returns 50.

ROUNDUP or ROUNDDOWN

Syntax: `ROUNDUP(Number; Count); ROUNDDOWN(Number; Count)`

These functions set the direction of rounding. If you are one of those contractors who bills a full hour for any fraction of an hour you work, you always want to round up so you don’t lose any money. Conversely, you might round down to give a slight discount to a long-established customer. As with the ROUND function, the *Count* argument sets the number of decimal places. With negative numbers, these functions treat UP as away from zero and DOWN as towards zero.

TRUNC

Syntax: `TRUNC(Number; Count)`

This function works the same as ROUNDDOWN, rounding positive and negative numbers to the next integer in the direction towards zero.

INT

Syntax: `INT(Number)`

For positive numbers, the INT function behaves like ROUNDDOWN and TRUNC, rounding to the next smallest integer. For negative numbers, it behaves differently, rounding away from zero. INT(-24.4) returns -25.

ODD and EVEN

Syntax: `ODD(Number); EVEN(Number)`

These functions round to the next odd or even number in the direction away from zero. ODD(23.5) returns 25. ODD(-23.5) returns -25.

CEILING and FLOOR

Syntax: `CEILING(Number; Significance; Mode);`
`Floor(Number; Significance; Mode)`

For positive numbers, these functions round numbers up (CEILING) or down (FLOOR) to the nearest multiple of the *Significance* argument, and the *Mode* argument has no effect. For negative numbers, if *Mode* is zero, CEILING rounds towards zero, and FLOOR rounds away from zero. For any other value of *Mode*, the direction of rounding negative numbers is reversed. The sign of *Significance* must match the sign of *Number* in all cases.

MROUND

Syntax: `MROUND(number; Multiple)`

MROUND rounds numbers to the nearest multiple of the *Multiple* argument.

One last point is worth mentioning: If you are working with more than two decimal places, don't be surprised if you don’t see the same number of decimal places on the spreadsheet as you do on the function wizard. If you don’t, the reason is that **Tools > Options > OpenOffice Calc > Calculate > Decimal Places** defaults to 2. Change the number of decimal places, and, if necessary, uncheck the **Precision as shown** box on the same page, and the spreadsheet will display as expected.

Function with arguments	***Result***	***Comment***
ROUND(Number; Count)		
=ROUND(123.456; 0)	123	
=ROUND(123.456; 2)	123.46	
=ROUND(123.456; -1)	120	
=ROUND(-123.456; 0)	-123	
=ROUND(-123.456; 2)	-123.46	
=ROUND(-123.456; -1)	-120	
ROUNDUP(Number; Count)		
=ROUNDUP(123.456;0)	124	
=ROUNDUP(123.454;2)	123.46	
=ROUNDUP(123.456;-1)	130	
=ROUNDUP(-123.456;0)	-124	"Up" is away from zero
=ROUNDUP(-123.454;2)	-123.46	"Up" is away from zero
=ROUNDUP(-123.456;-1)	-130	"Up" is away from zero
ROUNDDOWN(Number; Count)		
=ROUNDDOWN(123.456;0)	123	
=ROUNDDOWN(123.456;2)	123.45	
=ROUNDDOWN(126.456;-1)	120	
=ROUNDDOWN(-123.456;0)	-123	"Down" is towards zero
=ROUNDDOWN(-123.456;2)	-123.45	"Down" is towards zero
=ROUNDDOWN(-123.456;-1)	-120	"Down" is towards zero
TRUNC(Number; Count)		
=TRUNC(123.456;0)	123	
=TRUNC(123.456;2)	123.45	
=TRUNC(126.456;-1)	120	
=TRUNC(-123.456;0)	-123	Moves less negative. Compare to INT()
=TRUNC(-123.456;2)	-123.45	
=TRUNC(-123.456;-1)	-120	
INT(Number)		
=INT(123.456)	123	
=INT(-123.456)	-124	Moves more negative. Compare to TRUNC()
ODD(Number)		

=ODD(123.456)	125	Rounds up
=ODD(-123.456)	-125	Rounds away from zero
EVEN(Number)		
=EVEN(122.456)	124	Rounds up
=EVEN(-122.456)	-124	Rounds away from zero
CEILING(Number; Significance; Mode)		
=CEILING(77;5;0)	80	Mode argument has no effect on Positive Numbers
=CEILING(77;5;1)	80	
=CEILING(-77;-5;0)	-75	Mode argument changes the direction of rounding.
=CEILING(-77;-5;1)	-80	
FLOOR(Number; Significance; Mode)		
=FLOOR(77;5;0)	75	Mode argument has no effect on Positive Numbers
=FLOOR(77;5;1)	75	
=FLOOR(-77;-5;0)	-80	Mode argument changes the direction of rounding.
=FLOOR(-77;-5;1)	-75	
MROUND(number; multiple)		
=MROUND(15.3;5)	15	
=MROUND(-15.3;5)	-15	

Table 5: Examples of positive and negative numbers and rounding functions

Using Regular Expressions in Functions

A number of functions in Calc allow the use of regular expressions: SUMIF, COUNTIF, MATCH, SEARCH, LOOKUP, HLOOKUP, VLOOKUP, DCOUNT, DCOUNTA, DSUM, DPRODUCT, DMAX, DMIN, DAVERAGE, DSTDEV, DSTDEVP, DVAR, DVARP, DGET.

Whether or not regular expressions are used is selected on the **Tools > Options > OpenOffice Calc > Calculate** dialog.

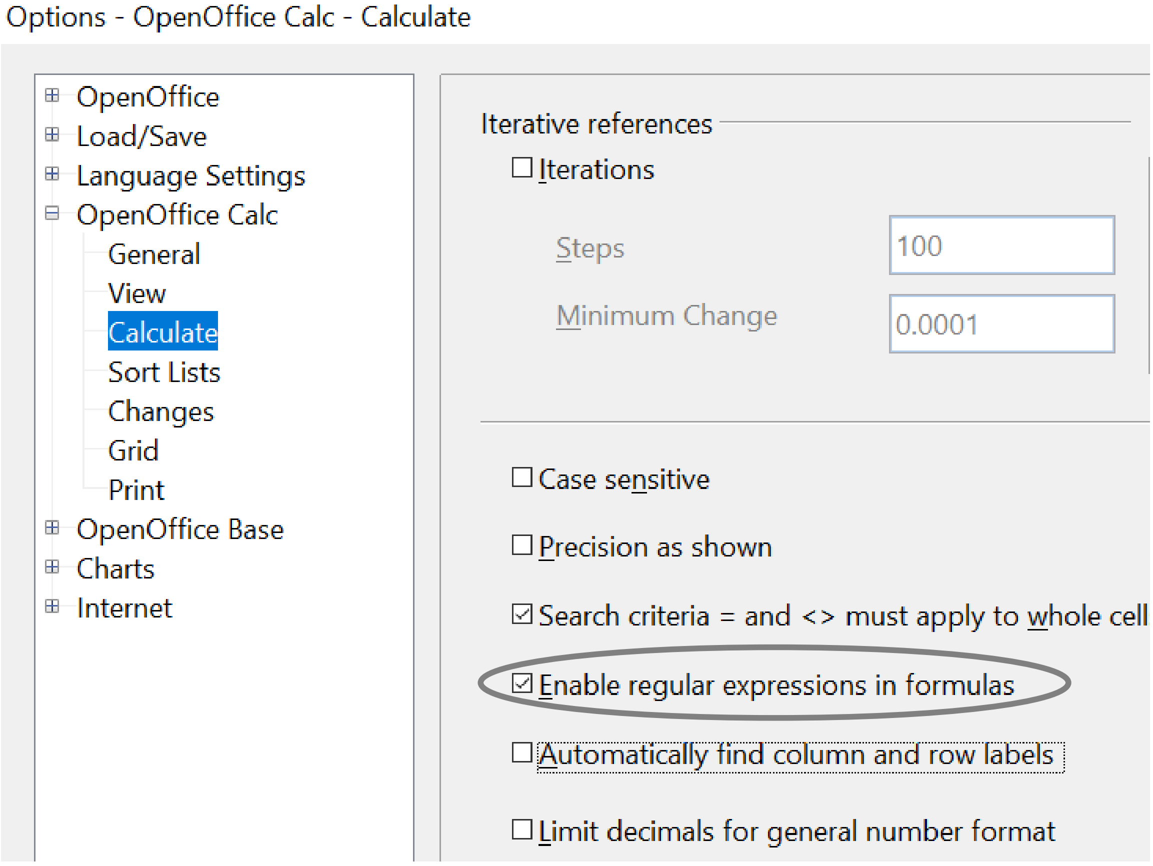


Figure 24: Enabling regular expressions in formulas

For example =COUNTIF(A1:A6;"r.d") with **Enable regular expressions in formulas** selected will count cells in A1:A6 which contain *red* and *ROD*.

Additionally, if **Search criteria = and <> must apply to whole cells** is *not* selected, then *Fred*, *bride*, and *Ridge* will also be counted. If that setting *is* selected, it can be overcome by wrapping the expression thus: =COUNTIF(A1:A6;".*r.d.*").

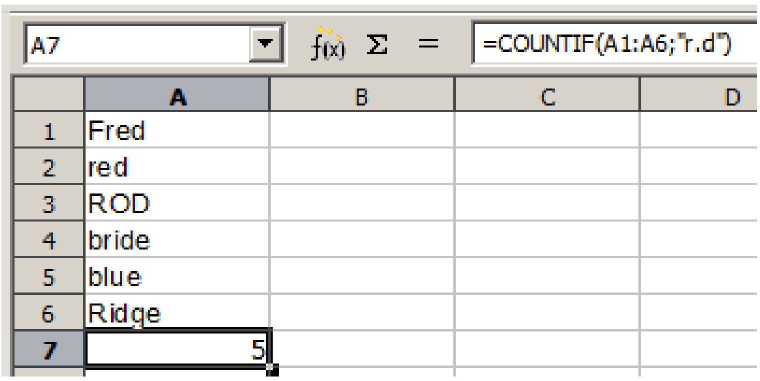


Figure 25: Using the COUNTIF function

Note	Regular expressions will not work in simple comparisons. For example, A1="r.d" will always evaluate to FALSE if A1 contains *red*, even if regular expressions are enabled. It will only test as TRUE if A1 contains *r.d* (*r* then a dot then *d*). If you wish to test using regular expressions, try the COUNTIF function: COUNTIF(A1; "r.d") will return 1 or 0, interpreted as TRUE or FALSE in formulas like =IF(COUNTIF(A1; "r.d"); "hooray"; "boo").

Regular expression searches *within functions* are always case insensitive, irrespective of the setting of the **Case sensitive** checkbox on the dialog in Figure 24—so *red* and *ROD* will always be matched in the above example. This case-insensitivity also applies to the regular expression structures ([:lower:]) and ([:upper:]), which match characters irrespective of case.

Activating the **Enable regular expressions in formulas** option means all the above functions will require any regular expression special characters (such as parentheses) used in strings within formulas to be preceded by a backslash despite not being part of a regular expression. These backslashes will need to be removed if the setting is later deactivated.

Advanced Functions

Like other spreadsheet programs, OpenOffice Calc can be enhanced by user-defined functions or add-ins. You can set up user-defined functions either by using the Basic IDE or by writing separate add-ins or extensions.

The basics of writing and running macros is covered in Chapter 12 (Calc Macros). Macros can be linked to menus or toolbars for ease of operation or stored in template modules to make the functions available in other documents.

Calc Add-Ins are specialized office extensions that can extend the functionality of OpenOffice with newly built-in Calc functions. Writing add-ins requires knowledge of the C++ language and the OpenOffice SDK, and is for experienced programmers. More information is available on the OpenOffice wiki page at https://wiki.openoffice.org/wiki/Calc/Add-In/Simple_Calc_Add-In. Several extensions for Calc have been written, and these can be found on the extensions site at https://extensions.openoffice.org/. Refer to Chapter 14 (Setting up and Customizing Calc) for more details.